РЕФЕРАТ

Отчёт 78 с., 34 рис., 9 табл., 12 ист., 4 прил.

ИНФОРМИРОВАНИЕ ВОДИТЕЛЕЙ, ДЕФЕКТЫ ДОРОЖНОГО ПОКРЫТИЯ, КРАУДСОРСИНГ, МИКРОСЕРВИСНАЯ АРХИТЕКТУРА, ГЕОПРОСТРАНСТВЕННЫЕ ДАННЫЕ, POSTGIS, FASTAPI, REACT NATIVE, МОБИЛЬНОЕ ПРИЛОЖЕНИЕ

Выпускная квалификационная работа посвящена разработке системы информирования водителей о дефектах дорожного покрытия на основе краудсорсингового подхода.

Объект исследования: процесс сбора, обработки, модерации и распространения информации о дефектах дорожного покрытия.

Цель работы: разработать информационную систему для оперативного выявления, модерации, отображения на карте дефектов дорожного покрытия и своевременного оповещения участников движения.

Методы исследования: системный анализ предметной области; методы объектно-ориентированного программирования (Python, JavaScript); методы работы с геоданными; функциональное и нагрузочное тестирование.

Результаты работы: Проведён анализ существующих решений, выявлены их недостатки. Спроектирована микросервисная архитектура, включающая сервис управления доступом (FastAPI, PostgreSQL, Redis) и сервис управления дефектами (FastAPI, PostGIS, MinIO). Клиентское приложение реализовано на React Native с отображением карты на базе MapLibre GL и векторных тайлов PMTiles. Разработаны алгоритмы привязки дефектов к дорожной сети, пространственных запросов (поиск в радиусе и области экрана) и динамического оповещения водителей с учётом скорости и направления движения. Проведено тестирование: 42 модульных теста API успешно пройдены; нагрузочное тестирование показало время поиска дефектов 156 мс при 100 000 записей.

Выводы: Созданная система обеспечивает сбор сообщений о дефектах от пользователей, их модерацию, визуализацию на карте и оповещение водителей. Разработанная система может быть использована

для оперативного информирования водителей о дефектах дорог, а также в качестве платформы для взаимодействия граждан с дорожными службами.

СОДЕРЖАНИЕ

Введение 6

1 Аналитический обзор 10

1.1 Анализ предметной области 10

1.1.1 Классификация дефектов дорожного покрытия 10

1.1.2 Влияние дефектов дорожного покрытия на безопасность движения 11

1.1.3 Традиционные подходы к обнаружению дефектов дорог и информированию водителей о них 12

1.2 Обзор существующих программных решений 14

1.2.1 Яндекс.Карты 14

1.2.2 2ГИС 14

1.2.3 Waze 14

1.2.4 Проект «Карта убитых дорог» 14

1.2.5 Сравнительный анализ 14

1.3 Анализ технологий и инструментов, применимых для разработки 15

1.3.1 Анализ архитектурных подходов 15

1.3.2 Анализ веб-фреймворков для серверной части 17

1.3.3 Анализ обратных прокси и API-шлюзов 17

1.3.4 Анализ кроссплатформенных фреймворков для мобильных приложений 19

1.3.5 Анализ систем управления базами данных 20

1.3.6 Анализ объектных хранилищ 21

1.3.7 Анализ in-memory систем для кэширования и хранения сессий 22

1.3.8 Анализ инструментов мониторинга 24

1.4 Требования к разрабатываемой системе 24

1.4.1 Функциональные требования 24
1.4.2 Нефункциональные требования 26
1.5 Выводы 28
2 Проектная часть 30
2.1 Архитектура системы 30
2.1.1 Контекст взаимодействия 30
2.1.2 Компоненты системы 30
2.1.3 Модель данных 32
2.2 Описание ключевых алгоритмов 36
2.2.1 Загрузка и отображение дефектов на карте 36
2.2.2 Оповещение водителя при приближении к дефекту 39
2.3 Выводы 41
3 Глава 3. Реализация и тестирование 43
3.1 Реализация сервиса управления доступом 43
3.2 Реализация сервиса управления дефектами 44
3.3 Реализация клиентского приложения 45
3.4 Тестирование системы 47
3.4.1 Тестирование API 47
3.4.2 Тестирование пространственных запросов 48
3.4.3 Тестирование клиентского приложения 48
3.4.4 Результаты тестирования 49
3.5 Выводы 49
Заключение 50
Список использованных источников 53
Приложение А Алгоритмы сервиса управления доступом 54
Приложение Б Алгоритмы сервиса управления дефектами 57
Приложение В Swagger-документация API 60

Приложение Г Интерфейс мобильного приложения 63

ВВЕДЕНИЕ

На сегодняшний день не существует централизованной системы оперативного информирования водителей о вновь возникших дефектах дорожного покрытия. Дорожные службы фиксируют повреждения в рамках плановых осмотров или по обращениям, но эти данные, как правило, не доводятся до участников движения. В результате водитель узнаёт о яме или выбоине чаще всего только в момент столкновения с ней. Между тем даже небольшой дефект может стать причиной потери управления автомобилем, особенно при высокой скорости, в тёмное время суток или в условиях плохой погоды.

В этой ситуации особенно ценным становится опыт самих участников движения: водители и пешеходы первыми обнаруживают новые дефекты. Если предоставить им простой и надёжный способ сообщить об угрозе, а другим участникам получить это предупреждение до попадания в зону риска, можно снизить вероятность аварий. Краудсорсинговая модель позволяет реализовать именно такой подход: превратить пользователей в распределённую сеть наблюдателей, чьи проверенные сообщения становятся основой для персонализированных и своевременных уведомлений. Такая система не требует установки дорогостоящих датчиков и не зависит от скорости бюрократических процедур, она работает за счёт активности сообщества и современных технологий.

Таким образом, разработка платформы, обеспечивающей оперативный обмен информацией о дорожных дефектах между пользователями, представляет собой не только технически реализуемое, но и социально значимое решение, направленное на снижение аварийности и повышение предсказуемости дорожной среды.

В сложившейся ситуации наблюдается противоречие между острой потребностью участников дорожного движения в оперативной и достоверной информации о дефектах дорожного покрытия и отсутствием эффективной краудсорсинговой системы, обеспечивающей сбор, верификацию, модерацию и мгновенное распространение таких данных в реальном времени. Разрешение данного противоречия составляет проблему настоящего исследования.

Объектом исследования является процесс сбора, обработки, модерации и распространения информации о дефектах дорожного покрытия с использованием краудсорсингового подхода.

Предмет исследования — методы, алгоритмы и программные средства для реализации системы информирования участников дорожного движения о дефектах дорог, включая серверную часть на основе микросервисной архитектуры, клиентское мобильное приложение и механизмы отображения на карте.

Цель работы — разработать информационную систему, основанную на краудсорсинговом принципе сбора данных от пользователей, для оперативного выявления, модерации, отображения на карте дефектов дорожного покрытия и своевременного оповещения участников движения.

Для достижения поставленной цели необходимо решить следующие задачи:

1. провести анализ существующих решений и требований: изучить аналоги систем мониторинга дорожных дефектов, определить функциональные и нефункциональные требования к системе;
2. спроектировать архитектуру системы: разработать модель взаимодействия компонентов (сервис управления доступом, сервис управления дефектами, клиентское приложение), определить структуру хранения данных;
3. реализовать серверную часть системы: разработать сервисы, обеспечивающие управление доступом (идентификация, аутентификация, авторизация) и управление дефектами (создание, модерация, геопоиск);
4. реализовать клиентское мобильное приложение: разработать приложение для мобильных устройств, обеспечивающее отображение дефектов на карте, добавление новых дефектов с фотографиями и геолокацией, а также получение оповещений;
5. провести тестирование и подготовить документацию: выполнить тестирование разработанной системы, составить техническое описание, подготовить репозиторий с кодом и инструкцию по развёртыванию.

В работе используются следующие методы исследования:

1. системный анализ для изучения предметной области и существующих аналогов;
2. методы объектно-ориентированного программирования;
3. методы работы с геоданными;
4. функциональное тестирование программного обеспечения.

Теоретическая значимость работы заключается в систематизации подходов к построению краудсорсинговых систем мониторинга дорожного покрытия, определении требований к их архитектуре и функциональности, а также в выявлении особенностей обработки геоданных в контексте задач отображения на карте и информирования участников дорожного движения.

Практическая значимость работы состоит в разработке и реализации серверной части и клиентского мобильного приложения. Разработанная система может быть использована для оперативного информирования водителей о дефектах дорог, а также внедрена в качестве платформы для взаимодействия граждан с дорожными службами и органами контроля.

В результате выполнения работы будет разработан рабочий прототип информационной системы, основанной на краудсорсинге, который обеспечивает:

1. сбор сообщений от пользователей о вновь выявленных дефектах дорожного покрытия с привязкой к географическим координатам;
2. модерацию поступающих данных для исключения дубликатов и ложных сообщений;
3. отображение подтверждённых дефектов на интерактивной карте в мобильном приложении;
4. оповещение пользователей, находящихся в непосредственной близости от зарегистрированного дефекта.

Система будет реализована в виде мобильного приложения с адаптивным интерфейсом и не будет зависеть от внешних дорогостоящих компонентов.

Текст выпускной квалификационной работы состоит из введения, трёх глав, заключения, списка использованных источников и приложений. Первая глава посвящена анализу существующих решений в области

мониторинга дорожных дефектов, сравнительному анализу аналогов и определению требований к системе. Вторая глава содержит проектирование архитектуры системы, модели данных и ключевых алгоритмов. Третья глава описывает практическую реализацию всех компонентов системы и результаты тестирования. В заключении подводятся итоги работы и определяются направления дальнейшего развития.

1 Аналитический обзор

1.1 Анализ предметной области

1.1.1 Классификация дефектов дорожного покрытия

Для корректного проектирования системы информирования водителей о дефектах дорожного покрытия необходимо опираться на нормативную базу, регламентирующую эксплуатационное состояние автомобильных дорог. Основным действующим документом в Российской Федерации является ГОСТ Р 50597-2017 «Дороги автомобильные и улицы. Требования к эксплуатационному состоянию, допустимому по условиям обеспечения безопасности дорожного движения. Методы контроля» [1].

Согласно данному стандарту, **дефект** определяется как несоответствие транспортно-эксплуатационных показателей конструктивных элементов дорог и улиц, дорожных сооружений и элементов обустройства требованиям настоящего стандарта.

В рамках исследования рассматриваются следующие виды дефектов дорожного покрытия, наиболее часто встречающиеся на российских дорогах и представляющие опасность для участников движения.

Трещины являются наиболее распространёнными деформациями покрытий. В зависимости от ориентации и характера распространения различают:

- **Продольная трещина** — трещина, идущая параллельно оси дороги. Образуется вследствие усталости материала или температурных деформаций.
- **Поперечная трещина** — трещина, пересекающая ось дороги. Возникает из-за резких перепадов температур или циклических нагрузок.
- **Сетка трещин** — множественные взаимосвязанные трещины, образующие замкнутые ячейки. Свидетельствует об усталостном разрушении дорожной одежды.
- **Заделанная трещина** — трещина, прошедшая ремонт. Требует контроля качества выполненного ремонта.

Локальные разрушения покрытия

- **Выбоина** — местное разрушение дорожного покрытия, имеющее вид углубления с резко очерченными краями. Согласно приложению А ГОСТ Р 50597-2017, является наиболее опасным видом дефекта.
- **Колейность** — деформация покрытия с образованием продольных углублений по полосам наката. Возникает из-за остаточных деформаций в покрытии или основании.

Дефекты разметки и элементов обустройства

- **Смытая разметка пешеходного перехода** — нечёткая или частично отсутствующая горизонтальная разметка типа «зебра».
- **Смытая разметка полос** — нечёткая или стёршаяся разметка, разделяющая полосы движения.
- **Люк колодца** — выступающий или проваливающийся относительно уровня покрытия люк инженерных коммуникаций.

Ремонтные вмешательства

- **Заплатка** — участок покрытия, отремонтированный методом карты. Часто имеет неровности на стыках с существующим покрытием.

Представленная классификация охватывает основные виды дефектов, с которыми сталкиваются участники дорожного движения. Понимание особенностей каждого типа дефекта необходимо для определения требований к системе сбора и отображения информации, а также для разработки сценариев оповещения водителей в зависимости от типа обнаруженного дефекта.

1.1.2 Влияние дефектов дорожного покрытия на безопасность движения

В статье «Влияние состояния дорожного полотна и дорожных условий на безопасность дорожного движения» авторы отмечают, что одной из главных причин дорожно-транспортных происшествий на участках с неудовлетворительным состоянием дорожного покрытия является столкновение автомобилей при резком торможении перед неровностью, а

также столкновения при попытке объезда неровностей с выездом на полосу встречного движения [2].

По данным авторов, за 2021 год зарегистрировано почти 53 тысячи ДТП из-за неудовлетворительного состояния дорог, в которых погибло 25 тысяч и ранено 68 тысяч человек. Неровность покрытия, по данным ГИБДД, является причиной 14% ДТП, связанных с неблагоприятными дорожными условиями.

Неровности дорожного полотна вызывают колебания трансмиссии автомобиля, что может привести к потере управляемости. Колебания прицепов и полуприцепов увеличивают динамический коридор движения и вероятность столкновения. Кроме того, наличие неровностей повышает утомляемость водителей, отвлекает их внимание от дороги и снижает пропускную способность.

Значительное число ДТП в тёмное время суток объясняется резким ухудшением условий зрительного восприятия, а также возможным ослеплением водителей отражённым светом фар от воды, заполняющей неровности.

Таким образом, состояние дорожного полотна напрямую влияет на безопасность движения, что обосновывает необходимость создания системы оперативного информирования водителей о дефектах покрытия.

1.1.3 Традиционные подходы к обнаружению дефектов дорог и информированию водителей о них

Обнаружение дефектов дорожного покрытия и информирование водителей о них традиционно осуществляется следующими методами.

Выявление дефектов производится с помощью:

- **Плановых осмотров и передвижных лабораторий** — дорожные службы проводят регулярные объезды закреплённых участков, фиксируя обнаруженные дефекты в актах. Для более точной диагностики используются специализированные автомобили-диагносты с измерительным оборудованием (профилометры, лазерные сканеры). Недостатком является низкая периодичность и

высокие затраты, из-за чего водитель узнаёт о дефекте только при непосредственном столкновении с ним.

- **Обращений граждан** — заявки от водителей и пешеходов о замеченных дефектах через государственные порталы и приложения. Однако обработка заявок занимает длительное время, а информация о подтверждённых дефектах не передаётся другим водителям.

Информирование водителей об уже выявленных дефектах осуществляется с помощью:

- **Дорожных знаков** — временные знаки «Неровная дорога» (1.16) устанавливаются на участках с известными дефектами, но не содержат информации о точном местоположении и степени опасности каждого дефекта.
- **Навигационных приложений** — Waze позволяет водителям отмечать выбоины как отдельный тип дорожной опасности. Яндекс.Карты и 2ГИС предоставляют возможность отмечать «дорожные работы», «ДТП» и «камеры», но не поддерживают классификацию конкретных дефектов покрытия (трещины, колейность, смытая разметка и т.п.). Кроме того, данные отметки, как правило, не проходят модерацию и не интегрированы в единую базу дефектов.

Основными недостатками традиционных подходов являются:

- неоперативность — между обнаружением дефекта и информированием водителя проходят дни или недели;
- неполнота данных — отсутствует единая база дефектов, доступная всем водителям;
- отсутствие обратной связи — водитель не может узнать, устранён ли обнаруженный им дефект;
- высокие затраты на специальное оборудование для выявления дефектов.

Таким образом, традиционные методы не обеспечивают водителя оперативной и достоверной информацией о вновь возникших дефектах.

1.2 Обзор существующих программных решений

1.2.1 Яндекс.Карты

Яндекс.Карты позволяют пользователям отмечать «дорожные работы», «ДТП» и «камеры». Возможности для отметки конкретных дефектов (выбоины, трещины, колейность, дефекты разметки) отсутствуют. Информация о состоянии покрытия передаётся косвенно — через отметки о ремонтных работах.

1.2.2 2ГИС

2ГИС предоставляет возможность отмечать «дорожные работы» и «ДТП», но не поддерживает классификацию дефектов дорожного покрытия. Информация о выбоинах, трещинах или смытой разметке отсутствует.

1.2.3 Waze

Waze позволяет водителям отмечать выбоины (Pothole) и другие дорожные опасности (Hazards). Сообщения проходят модерацию и отображаются на карте для других водителей в реальном времени. Однако Waze не покрывает все типы дефектов, выделенные в настоящем исследовании (трещины, колейность, дефекты разметки и т.д.).

1.2.4 Проект «Карта убитых дорог»

Проект «Дорожная инспекция / Карта убитых дорог» позволяет фиксировать отдельные дефекты: ямы, провалившиеся люки, стёршуюся разметку, неработающие светофоры, сломанные знаки. Заявки проходят обязательную модерацию перед добавлением в общую базу. Однако проект ориентирован на взаимодействие с государственными органами, а не на оперативное информирование водителей. Отсутствует система оповещений при приближении к дефекту.

1.2.5 Сравнительный анализ

Проведённый анализ позволяет сделать следующие выводы:

- Коммерческие навигационные сервисы (Яндекс.Карты, 2ГИС) не поддерживают детальную классификацию дефектов дорожного покрытия.
- Waze выделяется наличием прямой отметки опасностей, но не покрывает все типы дефектов, рассматриваемые в работе.
- «Карта убитых дорог» предоставляет детальную классификацию и модерацию, но ориентирована на госорганы, а не на оповещение водителей.

Таким образом, существующие решения не обеспечивают одновременно:

- детальную классификацию различных типов дефектов;
- оперативное оповещение водителей;
- модерацию поступающих сообщений.

Выявленные недостатки подтверждают актуальность разработки системы, сочетающей все перечисленные функции.

1.3 Анализ технологий и инструментов, применимых для разработки

Для обоснования выбора технологического стека разрабатываемой системы необходимо провести анализ существующих архитектурных подходов, веб-фреймворков, систем управления базами данных и инструментов мониторинга, которые могут быть использованы при реализации проекта.

1.3.1 Анализ архитектурных подходов

При проектировании программных систем выделяют два основных архитектурных подхода: монолитная архитектура и микросервисная архитектура. Подробное описание этих подходов и принципов их применения представлено в работах Ричардсона и Ньюмена [3,4].

Монолитная архитектура представляет собой единое приложение, включающее все компоненты системы. Все модули работают в рамках одного процесса и используют общую базу данных. Преимуществами монолитного подхода являются простота разработки на начальном этапе,

отсутствие дополнительных затрат на межсервисное взаимодействие и упрощённое развёртывание. Недостатки: сложность масштабирования (только всего приложения целиком), низкая гибкость (изменение одного модуля требует пересборки всего приложения), высокая связанность компонентов.

Микросервисная архитектура предполагает разделение системы на несколько независимых сервисов, каждый из которых отвечает за свою функциональную область. Сервисы взаимодействуют через сеть и могут развёртываться независимо. Преимущества: независимое масштабирование, независимость разработки и развёртывания, устойчивость к отказам. Недостатки: повышенная сложность организации взаимодействия между сервисами, дополнительные затраты на сетевые вызовы.

В статье «Сравнительный анализ архитектурных подходов к разработке REST API для высоконагруженных систем» отмечается, что выбор между монолитной и микросервисной архитектурой зависит от требований к масштабируемости и нагрузке системы [5]. Для высоконагруженных систем с необходимостью независимого масштабирования компонентов предпочтительнее микросервисный подход. Результаты нагрузочного тестирования показали, что монолитная архитектура демонстрирует лучшие показатели задержки в пределах одного сервера, в то время как микросервисная архитектура выигрывает в распределённых сценариях и при необходимости горизонтального масштабирования.

Характеристики рассматриваемых подходов приведены в таблице 1.

Таблица 1 — Сравнение архитектурных подходов

Критерий	Монолитная	Микросервисная
Масштабирование	Только целиком	Независимое
Развёртывание	Единое	Независимое
Связанность компонентов	Высокая	Низкая
Устойчивость к отказам	Низкая	Высокая
Технологическая гибкость	Отсутствует	Присутствует
Сложность разработки	Низкая	Высокая

1.3.2 Анализ веб-фреймворков для серверной части

Для реализации серверной части рассматриваются следующие варианты: FastAPI (Python), Django REST Framework (Python), ASP.NET Core (C#), NestJS (TypeScript), Spring Boot (Java).

В статье «Сравнительный анализ архитектурных подходов к разработке REST API для высоконагруженных систем» проведено нагрузочное тестирование перечисленных фреймворков [5]. В ходе тестирования измерялись средняя задержка, пропускная способность, p95 задержка и процент ошибок. Результаты представлены в таблице 2.

Таблица 2 — Результаты нагрузочного тестирования веб-фреймворков

Фреймворк	Средняя задержка, мс	Пропускная способность, req/s	p95 задержка, мс	Ошибки, %
FastAPI	43	1900	87	0,2
ASP.NET Core	52	1740	102	0,4
NestJS	61	1320	123	0,5
Django REST	128	730	246	1,1
Spring Boot	97	960	205	0,9

Как показано в таблице, наилучшие результаты по всем метрикам демонстрируют FastAPI и ASP.NET Core. Эти фреймворки обеспечивают минимальную задержку, высокую пропускную способность и низкий процент ошибок, что делает их предпочтительными для реализации высоконагруженных систем.

1.3.3 Анализ обратных прокси и API-шлюзов

В микросервисной архитектуре для маршрутизации запросов, балансировки нагрузки и ограничения частоты запросов используются обратные прокси-серверы или API-шлюзы.

Рассматриваются следующие решения:

1. **NGINX** — высокопроизводительный веб-сервер и обратный прокси с поддержкой балансировки нагрузки (round-robin, least connections, IP hash), ограничения частоты запросов (rate limiting), TLS-

терминации и кэширования. Обладает низким потреблением ресурсов и высокой пропускной способностью. Имеет как коммерческую (NGINX Plus), так и open-source версию.

2. **HAProxy** — специализированный программный балансировщик нагрузки и прокси-сервер с широкими возможностями маршрутизации и мониторинга состояния серверов (health checks). Поддерживает множество алгоритмов балансировки и используется в высоконагруженных системах.
3. **Traefik** — современный обратный прокси с интеграцией в экосистему контейнеризации (Docker, Kubernetes). Автоматически обнаруживает сервисы и обновляет конфигурацию маршрутизации. Поддерживает автоматическое получение TLS-сертификатов через Let's Encrypt.
4. **Envoy** — высокопроизводительный прокси-сервер, разработанный для облачных и микросервисных архитектур. Используется в качестве sidecar-прокси в сервисных сетках (service mesh) на базе Istio.

Сравнение обратных прокси и API-шлюзов представлено в таблице 3.

Таблица 3 — Сравнение обратных прокси и API-шлюзов

Критерий	NGINX	HAProxy	Traefik	Envoy
Балансировка нагрузки	Да	Да	Да	Да
Rate limiting	Да	Да	Да	Да
TLS-терминация	Да	Да	Да	Да
Интеграция с Docker	Ограниченная	Нет	Встроенная	Частичная
Автоматическое обнаружение сервисов	Нет	Нет	Да	Да
Простота настройки	Средняя	Средняя	Высокая	Низкая
Open Source	Да (основная версия)	Да	Да	Да

На основе проведённого анализа можно сделать следующие выводы:

1. **NGINX** — наиболее распространённое и проверенное решение, обеспечивающее высокую производительность и низкое потребление ресурсов. Подходит для большинства задач маршрутизации и балансировки нагрузки.
2. **Traefik** — удобный выбор для сред с активным использованием контейнеров, благодаря встроенной интеграции с Docker и автоматической настройке маршрутов.
3. **HAProxy** — специализированное решение для высоконагруженных систем с требованиями к отказоустойчивости.
4. **Envoy** — используется преимущественно в составе service mesh для микросервисных архитектур.

1.3.4 Анализ кроссплатформенных фреймворков для мобильных приложений

Для реализации клиентского мобильного приложения рассматриваются следующие варианты: React Native, Flutter и нативная разработка (Kotlin для Android + Swift для iOS).

React Native — фреймворк, разработанный компанией Meta, позволяющий создавать кроссплатформенные мобильные приложения с использованием JavaScript/TypeScript и React. Приложения компилируются в нативный код, что обеспечивает высокую производительность. React Native имеет огромное сообщество и богатую экосистему библиотек для работы с картами (react-native-maps), геолокацией и push-уведомлениями.

Flutter — фреймворк от Google, использующий язык Dart и собственный движок Skia для рендеринга. Обеспечивает отличную производительность и богатый набор встроенных виджетов. Однако сообщество и количество сторонних библиотек меньше, чем у React Native.

Нативная разработка предполагает создание отдельных приложений для iOS (Swift) и Android (Kotlin/Java). Обеспечивает максимальную производительность и доступ ко всем возможностям платформы. Основным недостатком является необходимость поддержки двух раздельных кодовых баз, что увеличивает время затрачиваемое на разработку.

Сравнение кроссплатформенных фреймворков представлено в таблице 4.

Таблица 4 — Сравнение кроссплатформенных фреймворков

Критерий	React Native	Flutter	Нативная
Кодовая база	Единая (JS/TS)	Единая (Dart)	Две раздельные
Производительность карт	Хорошая	Отличная	Отличная
Скорость разработки	Высокая	Высокая	Низкая
Размер сообщества	Огромный	Растущий	Огромный
Экосистема библиотек	Богатая	Развивающаяся	Огромная

1.3.5 Анализ систем управления базами данных

Для хранения геопространственных данных рассматриваются следующие варианты: PostgreSQL с расширением PostGIS, Microsoft SQL Server (с модулем Spatial), Oracle Spatial and Graph, MongoDB.

В статье «Сравнительный анализ систем управления базами данных при реализации распределенного хранилища геоданных» отмечается, что ключевыми критериями выбора СУБД для геоданных являются поддержка геопространственных типов, наличие пространственных индексов, производительность выполнения геозапросов и масштабируемость в распределённой среде [6].

Результаты проведённого сравнительного анализа представлены в таблице 5.

Таблица 5 — Сравнение систем управления базами данных

Критерий	PostgreSQL/ PostGIS	Microsoft SQL Server	Oracle Spatial	MongoDB
Поддержка стандартов OGC	Полная	Базовая	Полная	Частичная
Геопростран-ственные типы	Векторные, растры, 3D	Векторные	Векторные, растры, 3D	Растры, метаданные

Критерий	PostgreSQL/ PostGIS	Microsoft SQL Server	Oracle Spatial	MongoDB
Пространственные индексы	Да (GiST)	Да	Да (R-tree)	Ограниченно
Поддержка ACID	Да	Да	Да	Нет
Поддержка распределённых БД	Через Citus	Always On	Oracle RAC	Шардирование

На основе проведённого анализа можно сделать следующие выводы:

1. **PostgreSQL/PostGIS** является наиболее универсальной системой, обеспечивающей сбалансированное сочетание функциональности, открытости и масштабируемости. Поддерживает все типы геоданных (векторные, растровые, облака точек) и стандарты OGC.
2. **Oracle Spatial and Graph** — оптимальный выбор для корпоративных и государственных задач, где важны масштабируемость, отказоустойчивость и сложные топологические операции.
3. **Microsoft SQL Server** демонстрирует надёжную работу с векторными данными, хорошую масштабируемость и интеграцию с Windows-средой.
4. **MongoDB** эффективна для хранения метаданных и растровых файлов в микросервисных архитектурах, но не подходит для сложных пространственных запросов.

1.3.6 Анализ объектных хранилищ

Для хранения файлов, не являющихся структурированными данными, в современных распределённых системах используются объектные хранилища. В разрабатываемой системе требуется хранить фотографии дефектов, загружаемые пользователями при создании сообщения о дефекте.

Рассматриваются следующие объектные хранилища:

1. **Amazon S3 (Amazon Simple Storage Service)** — проприетарное облачное решение, являющееся стандартом де-факто для объектного хранения [7]. Предоставляет высокую доступность, масштабируемость и простой API для загрузки и скачивания файлов.
2. **MinIO** — открытая реализация S3-совместимого объектного хранилища, предназначенная для развёртывания в собственной инфраструктуре [8]. Полностью совместима с S3 API, что позволяет использовать стандартные клиентские библиотеки.
3. **Ceph** — распределённая файловая система с поддержкой объектного хранения, обеспечивающая высокую отказоустойчивость и масштабируемость [9]. Требует более сложной настройки по сравнению с MinIO.

Сравнение объектных хранилищ представлено в таблице 6.

Таблица 6 — Сравнение объектных хранилищ

Критерий	Amazon S3	MinIO	Ceph
Лицензия	Проприетарная	Open Source (AGPLv3)	Open Source (LGPLv2)
Развёртывание	Облачное	Self-hosted	Self-hosted
Совместимость с S3 API	Нативная	Полная	Частичная (RADOS Gateway)
Сложность настройки	Низкая	Низкая	Высокая
Подходит для self-hosted	Нет	Да	Да

1.3.7 Анализ in-memory систем для кэширования и хранения сессий

В обзоре рынка in-memory систем управления базами данных отмечается, что такие системы хранят информацию непосредственно в оперативной памяти, что обеспечивает высокую скорость доступа и делает их идеальными для задач кэширования и управления сессиями [10]. Основные недостатки — ограниченный объём хранимых данных и энергозависимость.

Среди наиболее популярных in-memory решений можно выделить следующие:

1. **Redis** — высокопроизводительная система с открытым исходным кодом. Поддерживает различные структуры данных (строки, хеши, списки, множества), настройку времени жизни (TTL) для записей, а также сохранение данных на диск. На сегодняшний день Redis является наиболее распространённым решением для кэширования и хранения сессионных данных.
2. **Memcached** — высокопроизводительная многопоточная система кэширования. Поддерживает только базовые операции ключ-значение и не имеет механизма сохранения данных на диск.
3. **Tarantool** — СУБД с открытым исходным кодом, разработанная компанией VK. Позволяет хранить различные типы информации (таблицы, документы), имеет индексы, инструменты для шардирования, а также поддерживает сохранение данных на диск.

Сравнение основных характеристик in-memory систем представлено в таблице 7.

Таблица 7 — Сравнение in-memory систем управления базами данных

Характеристика	Memcached	Redis	Tarantool
Компания разработчика	Danga Interactive	Redis Labs	VK
Лицензия	Open Source (BSD)	Open Source (Redis Source Available License v2)	Open Source (BSD)
Открытый исходный код	Да	Да	Да
Тип базы данных	Ключ-значение	Ключ-значение	Мультидомальная, ключ-значение
Режимы хранения	Оперативная память	Оперативная память	Оперативная память
Транзакции	Нет	Да	Да

1.3.8 Анализ инструментов мониторинга

Для организации мониторинга системы рассматриваются следующие инструменты, широко применяемые в микросервисных архитектурах:

1. **Сбор метрик:** Prometheus — система с pull-моделью сбора метрик, которая регулярно опрашивает HTTP-эндпоинты сервисов. Для микросервисной архитектуры предпочтительнее pull-модель, так как она не требует настройки передачи данных из каждого экземпляра сервиса [11].
2. **Визуализация:** Grafana — наиболее распространённый инструмент для построения дашбордов, поддерживающий множество источников данных, включая Prometheus [12].
3. **Оповещение:** Alertmanager — встроенный компонент экосистемы Prometheus, позволяющий настраивать отправку уведомлений через Telegram, email, Webhook и другие каналы [11].

1.4 Требования к разрабатываемой системе

На основе проведённого анализа существующих решений и выявленных недостатков можно сформулировать требования к разрабатываемой системе информирования водителей о дефектах дорожного покрытия. Требования разделяются на функциональные и нефункциональные.

1.4.1 Функциональные требования

Подсистема управления доступом:

Система должна обеспечивать:

1. регистрацию новых пользователей с указанием имени, электронной почты и пароля;
2. аутентификацию пользователя по электронной почте и паролю с выдачей токена;
3. обновление сессии при истечении срока действия токена;
4. выход из системы с аннулированием текущей сессии;

5. выход из всех устройств с аннулированием всех активных сессий пользователя;
6. просмотр информации о текущем пользователе;
7. редактирование профиля пользователя;
8. создание, чтение, обновление и удаление ролей;
9. создание, чтение, обновление и удаление разрешений;
10. назначение ролей пользователям и отзыв ролей;
11. назначение разрешений ролям и отзыв разрешений;
12. проверку прав пользователя перед выполнением защищённых операций.

Подсистема управления дефектами:

Система должна обеспечивать:

1. создание дефекта с указанием типа, геометрического типа, координат, фотографий и описания;
2. привязку переданных координат к дорожной сети для повышения точности отображения;
3. получение информации о дороге для созданного дефекта;
4. просмотр списка дефектов с возможностью фильтрации по статусу и типу;
5. просмотр детальной информации о конкретном дефекте по его идентификатору;
6. редактирование собственных дефектов пользователем;
7. удаление собственных дефектов пользователем;
8. модерацию дефектов: подтверждение или отклонение с указанием причины;
9. автоматическое изменение статуса дефекта при подтверждении или отклонении;
10. поиск подтверждённых дефектов в заданном радиусе от указанных координат;
11. поиск подтверждённых дефектов внутри прямоугольной области для отображения на карте;
12. фильтрацию результатов поиска по типам дефектов и по минимальному уровню опасности.

Клиентское приложение:

Система должна обеспечивать пользователю возможность взаимодействия через мобильное приложение, которое позволяет:

1. регистрацию, вход и управление профилем;
2. отображение подтверждённых дефектов на интерактивной карте в реальном времени;
3. отображение информации о дефекте при нажатии на маркер;
4. фильтрацию отображаемых дефектов на карте по типу и уровню опасности;
5. создание нового дефекта с привязкой геопозиции, выбором типа, добавлением фотографий и описания;
6. просмотр и редактирование своих дефектов, отслеживание их статуса;
7. автоматическое получение уведомлений при приближении к подтверждённому дефекту.

Подсистема мониторинга:

Система должна обеспечивать:

1. сбор метрик о работе сервисов (время ответа, количество запросов, частота ошибок);
2. визуализацию собираемых метрик для анализа состояния системы;
3. оповещение администратора при возникновении критических ситуаций.

1.4.2 Нефункциональные требования

Производительность:

1. время ответа API на запросы получения дефектов не должно превышать 1-2 секунд при стандартной нагрузке;
2. время отклика приложения при отображении карты и дефектов должно обеспечивать плавный скроллинг и масштабирование без заметных задержек.

Безопасность:

1. система должна обеспечивать защиту персональных данных пользователей;
2. пароли пользователей должны храниться в хэшированном виде;
3. аутентификация должна осуществляться с использованием токенов с ограниченным сроком действия;
4. доступ к операциям должен быть разграничен в зависимости от роли пользователя.

Доступность:

1. система должна быть доступна для пользователей круглосуточно;
2. допустимое время недоступности сервера — не более 0,1% времени в месяц.

Удобство использования:

1. интерфейс приложения должен быть интуитивно понятным и не требовать специального обучения;
2. приложение должно быть адаптировано для использования на мобильных устройствах с различными размерами экрана;
3. процесс создания дефекта должен занимать минимальное время (не более 3-4 действий).

Масштабируемость:

1. горизонтальное масштабирование сервисов должно достигаться добавлением дополнительных экземпляров без изменения исходного кода;
2. архитектура должна позволять масштабировать компоненты независимо друг от друга в зависимости от нагрузки.

Надёжность:

1. система должна корректно обрабатывать ошибки ввода пользователя и сетевые сбои;
2. при отсутствии сетевого соединения клиентское приложение должно сохранять введённые данные для последующей отправки после восстановления связи.

1.5 Выводы

В результате аналитического обзора определены основные типы дефектов дорожного покрытия, подлежащие фиксации в системе: трещины (продольные, поперечные, сетка, заделанные), выбоины, колейность, дефекты разметки (пешеходные переходы, полосы движения), люки колодцев и заплатки. Установлено, что дефекты дорожного покрытия оказывают существенное влияние на безопасность движения — по данным статистики, только в 2021 году из-за неудовлетворительного состояния дорог произошло почти 53 тысячи ДТП, в которых погибло 25 тысяч и ранено 68 тысяч человек [2]. Традиционные подходы к обнаружению дефектов (плановые осмотры, передвижные лаборатории, обращения граждан) и информированию водителей (дорожные знаки, навигационные приложения) характеризуются неоперативностью, неполнотой данных и высокими затратами.

Обзор существующих программных решений — Яндекс.Карты, 2ГИС, Waze, «Карта убитых дорог» — показал, что ни одно из них не сочетает детальную классификацию дефектов, оперативное оповещение водителей и модерацию сообщений. Проведённый анализ технологий и инструментов позволил выявить, что микросервисная архитектура является предпочтительной для высоконагруженных систем с необходимостью независимого масштабирования компонентов. Среди веб-фреймворков наилучшие показатели производительности демонстрируют FastAPI и ASP.NET Core. Среди кроссплатформенных фреймворков для мобильной разработки React Native обеспечивает оптимальное сочетание производительности, скорости разработки и размера сообщества. Для хранения геопространственных данных наиболее универсальным решением является PostgreSQL с расширением PostGIS, обеспечивающая полную поддержку геоданных и стандартов OGC. Среди объектных хранилищ для self-hosted развёртывания выделяется MinIO, предназначенный для хранения фотографий дефектов. Для хранения сессионных данных и кэширования наиболее функциональной системой является Redis. В качестве обратного прокси и балансировщика нагрузки наиболее распространённым решением является NGINX, поддерживающий также ограничение частоты запросов. Для мониторинга микросервисной архитектуры стандартным решением является связка Prometheus (сбор метрик), Grafana (визуализация) и Alertmanager (оповещение).

На основе выявленных недостатков существующих решений и проведённого анализа технологий сформулированы функциональные и нефункциональные требования к разрабатываемой системе, включая подсистемы управления пользователями, управления дефектами, клиентского приложения и мониторинга. В следующей главе на основе сформулированных требований будет представлено проектирование архитектуры системы и обоснование выбора конкретных технологий.

2 Проектная часть

2.1 Архитектура системы

2.1.1 Контекст взаимодействия

На верхнем уровне абстракции система взаимодействует с тремя типами пользователей: водитель, модератор и администратор.

1. **Водитель** — просматривает дефекты на карте, получает уведомления о приближении к дефектам, создаёт новые дефекты (статус pending), просматривает и редактирует свои дефекты.
2. **Модератор** — обладает расширенными правами, проверяет достоверность поступивших дефектов, подтверждает или отклоняет их.
3. **Администратор** — обладает полными правами, управляет пользователями (активация, деактивация), ролями и разрешениями.

Диаграмма контекста, иллюстрирующая взаимодействие системы с внешними сущностями, представлена на рисунке 1.

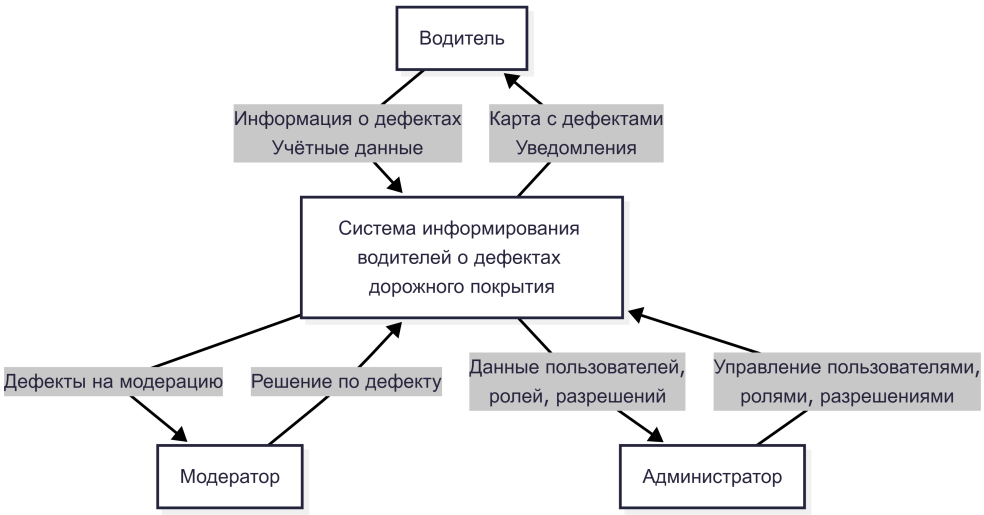

Рисунок 1 — Диаграмма контекста системы

2.1.2 Компоненты системы

Система состоит из следующих основных компонентов.

Клиентское приложение представляет собой мобильное и веб-приложение, обеспечивающее взаимодействие пользователя с системой. Для его реализации выбран фреймворк React Native на основе сравнительного анализа кроссплатформенных решений (таблица 4). Выбор обусловлен кроссплатформенностью (единая кодовая база для iOS и Android), богатой экосистемой библиотек для работы с картами и геолокацией, а также большим сообществом.

Обратный прокси (NGINX) выполняет функции балансировки нагрузки, маршрутизации запросов, ограничения частоты запросов и TLS-терминации. Выбор NGINX обусловлен его высокой производительностью, низким потреблением ресурсов и широкой распространённостью в высоконагруженных системах (таблица 6).

Сервис управления доступом отвечает за регистрацию, аутентификацию, управление сессиями, ролями и разрешениями. Для его реализации выбран веб-фреймворк FastAPI, который по результатам сравнительного анализа веб-фреймворков (таблица 2) демонстрирует наилучшие показатели производительности среди рассмотренных решений.

Сервис управления дефектами отвечает за создание, модерацию, хранение и поиск дефектов, включая пространственные запросы (поиск в радиусе, поиск в области экрана). Также реализован на FastAPI в силу тех же преимуществ производительности и удобства работы с геоданными.

БД пользователей (PostgreSQL) хранит данные аккаунтов пользователей, роли и права доступа. Выбор PostgreSQL обусловлен её надёжностью, поддержкой ACID-транзакций и широкой распространённостью в корпоративной среде.

БД кэша и сессий (Redis) обеспечивает временное хранение сессий и часто запрашиваемых данных, что позволяет снизить нагрузку на основную базу данных и ускорить ответы API. Redis выбран благодаря высокой производительности (хранение в оперативной памяти), поддержке TTL для автоматического удаления устаревших данных (таблица 5).

БД дефектов (PostgreSQL + PostGIS) хранит геоданные о дефектах дорожного покрытия. Выбор PostGIS (таблица 3) обусловлен полной поддержкой геометрических типов и пространственных индексов, что

критически важно для эффективного выполнения пространственных запросов.

Хранилище файлов (MinIO) используется для хранения фотографий дефектов и картографических тайлов (.pmtiles). Выбрано за открытую лицензию, полную совместимость с S3 API и возможность развёртывания в собственной инфраструктуре (таблица 4).

Агрегатор метрик (Prometheus) обеспечивает сбор и хранение временных рядов метрик от сервисов. Выбран как стандартное решение для микросервисных архитектур благодаря pull-модели сбора метрик.

Панель мониторинга (Grafana) обеспечивает визуализацию метрик и создание дашбордов для администратора. Выбрана в связке с Prometheus как наиболее распространённое решение.

Диаграмма контейнеров в нотации C4 представлена на рисунке 2.

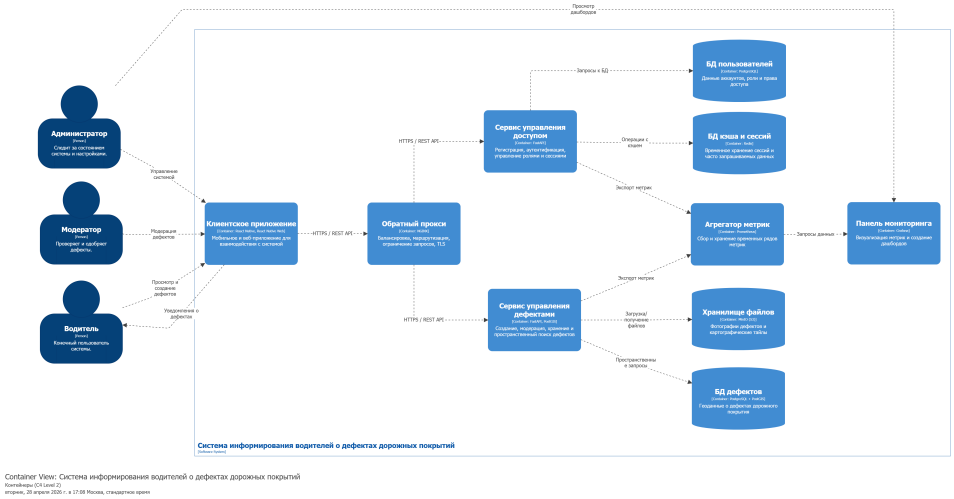

Рисунок 2 — Диаграмма контейнеров в нотации C4

2.1.3 Модель данных

2.1.3.1 Концептуальная модель

На концептуальном уровне система оперирует следующими основными сущностями:

1. **Пользователь** — субъект, взаимодействующий с системой (водитель, модератор, администратор).
2. **Сессия** — фиксирует факт входа пользователя в систему.

3. **Роль** — группа прав доступа (user, moderator, admin).
4. **Разрешение** — конкретное действие, разрешённое пользователю.
5. **Дефект** — основная сущность системы, описывающая повреждение дорожного покрытия.
6. **Дорога** — элемент дорожной сети, импортированный из OSM-данных.
7. **История входов** — аудит действий пользователя по входу в систему.

Связи между сущностями:

1. пользователь имеет несколько сессий и записей в истории входов;
2. пользователю могут быть назначены несколько ролей;
3. роль может содержать несколько разрешений;
4. пользователь создаёт и модерирует дефекты;
5. дефект привязан к дороге.

Концептуальная модель данных представлена на рисунке 3.

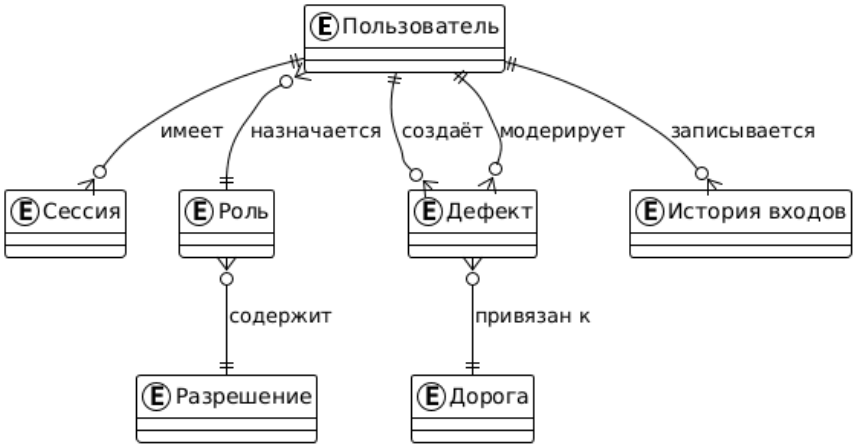

Рисунок 3 — Концептуальная модель данных

2.1.3.2 Детальная модель базы данных

Сервис управления доступом:

База данных сервиса управления доступом включает таблицы users, roles, permissions, user_roles, role_permissions и login_history. Связи между таблицами:

1. пользователи и роли — связь многие-ко-многим (через user_roles);

2. роли и разрешения — связь многие-ко-многим (через role_permissions);
3. пользователи и история входов — связь один-ко-многим.

Для хранения сессий используется Redis. Сессии хранятся с TTL, соответствующим сроку действия refresh-токена. ER-диаграмма сервиса управления доступом представлена на рисунке 4.

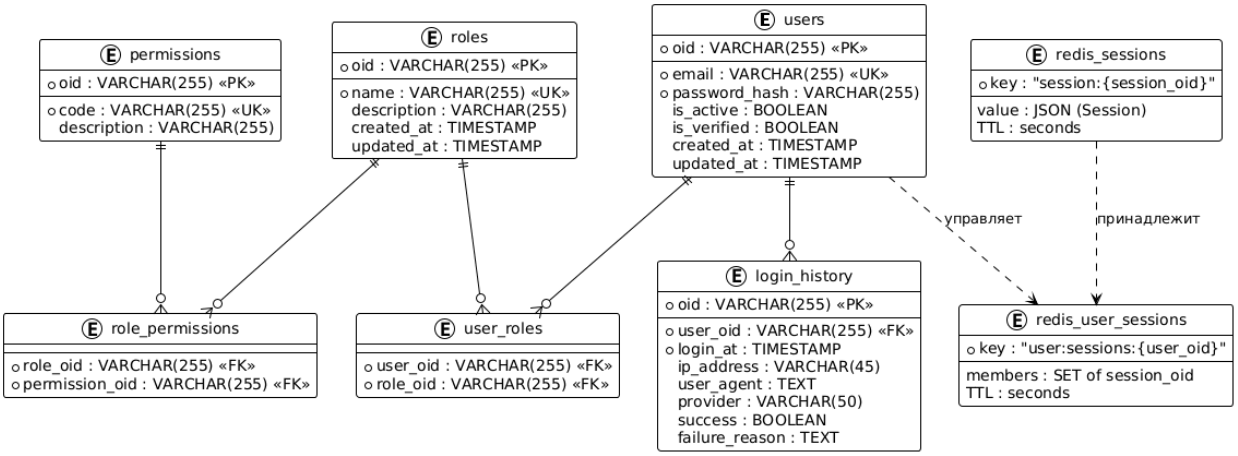

Рисунок 4 — ER-диаграмма сервиса управления доступом

Сервис управления дефектами:

База данных сервиса управления дефектами использует PostgreSQL с расширением PostGIS. Основная таблица **road_defects** содержит:

1. геометрические поля: original_geometry (исходные координаты), snapped_geometry (привязанные к дороге);
2. атрибуты дефекта: defect_type (тип), severity (уровень опасности), description (описание);
3. статусную модель: status (pending, approved, rejected, fixed), rejection_reason;
4. привязку к дороге: osm_way_id, road_name, road_class;
5. фотографии: поле photos (JSON-массив ссылок на файлы в MinIO);
6. аудиторскую информацию: created_by, created_at, moderated_by, moderated_at.

Для повышения производительности пространственных запросов созданы GiST-индексы на полях original_geometry и snapped_geometry, а также B-tree индекс на поле status.

Дорожная сеть импортируется из OSM-файлов (.osm.pbf) и хранится в таблицах, сгенерированных инструментом osm2pgsql:

1. **planet_osm_line** — содержит линейные объекты (дороги, реки, железные дороги);
2. **planet_osm_roads** — содержит только дороги (упрощённый вариант planet_osm_line);
3. **planet_osm_point** — содержит точечные объекты (POI, здания, достопримечательности);
4. **planet_osm_polygon** — содержит полигональные объекты (здания, парки, водоёмы);
5. **planet_osm_nodes**, **planet_osm_ways**, **planet_osm_rels** — служебные таблицы для хранения топологии OSM-данных.

Таблицы дорожной сети были импортированы один раз при инициализации системы и не изменяются в процессе её работы. Для пространственных запросов (привязка дефекта к дороге) используется таблица planet_osm_line с индексом на геометрическом поле.

ER-диаграмма сервиса управления дефектами представлена на рисунке 5.

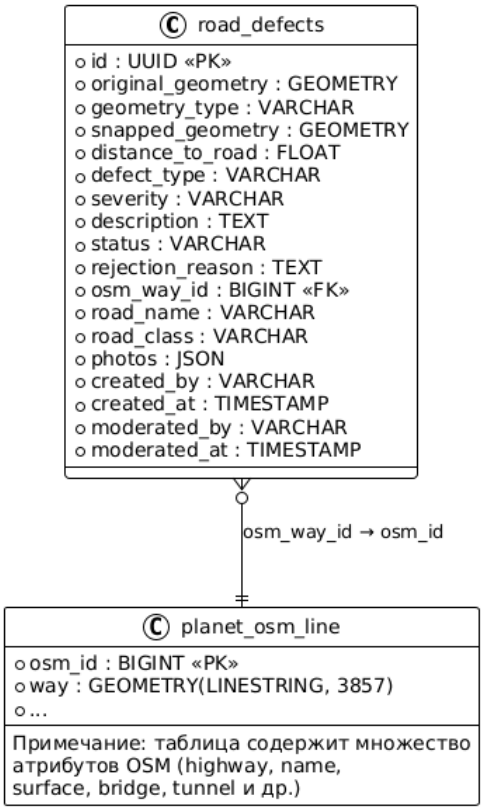

Рисунок 5 — ER-диаграмма сервиса управления дефектами

2.2 Описание ключевых алгоритмов

В данном разделе рассматриваются алгоритмы, определяющие специфическую функциональность системы: загрузку и отображение дефектов на карте, динамическое оповещение водителя с учётом скорости и направления движения. Остальные алгоритмы (аутентификация, обновление токенов, проверка прав доступа) вынесены в приложение А, а алгоритмы создания дефекта, поиска дефектов в радиусе и в видимой области карты — в приложение Б.

2.2.1 Загрузка и отображение дефектов на карте

Отображение дефектов на интерактивной карте является ключевой функцией клиентского приложения. Для обеспечения плавной работы при перемещении и масштабировании карты предлагается механизм интеллектуальной загрузки дефектов с разбиением на сетку и кэшированием.

Подготовка картографической подложки:

Карта реализуется на основе библиотеки MapLibre GL, работающей через WebView. В качестве картографической подложки предлагается использовать векторный тайловый набор в формате PMTiles, размещённый в объектном хранилище MinIO. PMTiles позволяет хранить всю карту в одном файле и получать необходимые тайлы по HTTP-запросам с поддержкой range-запросов, что даёт возможность работать с большими объёмами данных без полной загрузки файла.

Сеточная структура хранения дефектов:

Дефекты в клиентском приложении предлагается индексировать по сетке с размером ячейки 0,01 градуса (приблизительно 1,1 км). Для каждого дефекта вычисляется ключ ячейки по его координатам, что позволяет быстро находить дефекты в заданной области без полного перебора всего массива.

Загрузка дефектов по области экрана:

При первом открытии карты или при её перемещении срабатывает механизм отложенной загрузки (с задержкой 300 миллисекунд после остановки движения). Тип загрузки зависит от текущего масштаба:

1. При масштабе менее 12 загружаются все дефекты в пределах видимой области экрана одним запросом.
2. При масштабе 12 и более загружаются дефекты из расширенной области (видимая область плюс соседние ячейки), что обеспечивает плавное перемещение карты без «пустот» на границах.

Кэширование загруженных дефектов:

Для снижения количества запросов к серверу предлагается кэширование дефектов по ячейкам с временем жизни 10 минут. При загрузке новой области сначала проверяется наличие данных в кэше. Кэш ограничен по размеру — при превышении 100 ячеек удаляется самая старая.

Форматирование и отображение дефектов:

Полученные от сервера дефекты проходят этап форматирования: определяется геометрический тип (точка или линия), извлекаются координаты, каждому дефекту присваивается цвет в зависимости от уровня

опасности. Отрисовка выполняется через механизм источников и слоёв MapLibre GL с поддержкой кластеризации на малых масштабах.

Диаграмма последовательности загрузки и отображения дефектов представлена на рисунке 6.

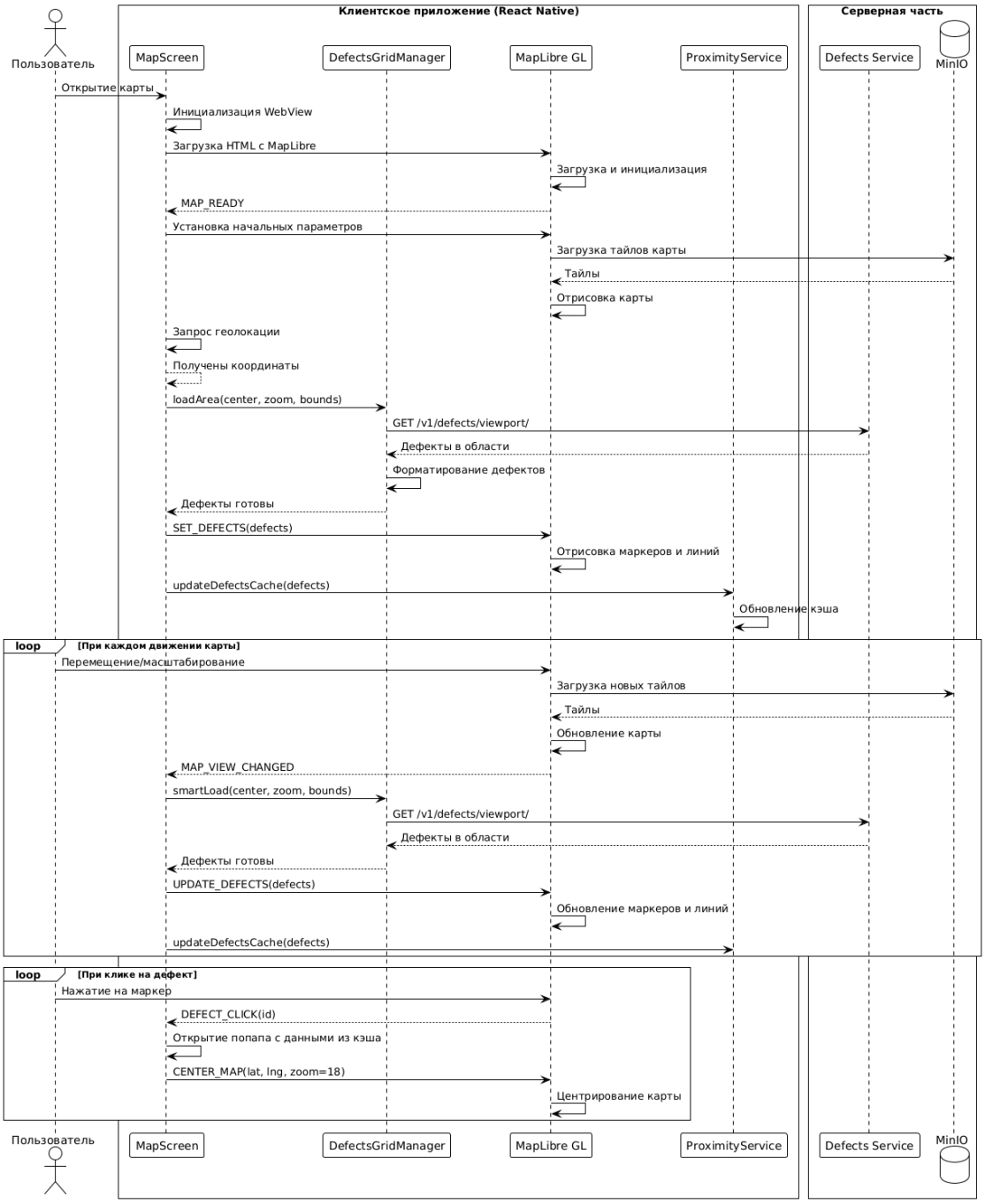

Рисунок 6 — Диаграмма последовательности загрузки и отображения дефектов

2.2.2 Оповещение водителя при приближении к дефекту

Оповещение предлагается реализовать на стороне мобильного приложения с периодической проверкой в фоновом режиме с учётом скорости и направления движения.

Определение параметров движения:

При каждом обновлении геопозиции (каждые 2 секунды) вычисляются текущая скорость и направление движения. Для расчёта направления используется формула азимута между двумя последовательными координатами:

$$\theta = \operatorname{atan2}(\sin(\Delta\lambda)\cos(\varphi_2), \cos(\varphi_1)\sin(\varphi_2) - \sin(\varphi_1)\cos(\varphi_2)\cos(\Delta\lambda)) \quad (1)$$

где φ_1, λ_1 — предыдущие координаты, φ_2, λ_2 — текущие координаты.

Расчёт динамического радиуса оповещения:

Радиус оповещения рассчитывается динамически в зависимости от скорости:

$$R = \min(\max(R_{\text{base}} + vk, R_{\min}), R_{\max}) \quad (2)$$

где $R_{\text{base}} = 80$ м — базовый радиус, v — скорость (км/ч), $k = 1,5$ — коэффициент, $R_{\min} = 50$ м, $R_{\max} = 300$ м.

Проверка направления движения:

При движении со скоростью более 5 км/ч дополнительно проверяется, находится ли дефект впереди автомобиля. Угол между направлением движения и направлением на дефект вычисляется по формуле:

$$\alpha = |\theta_{\text{defect}} - \theta| \quad (3)$$

где θ_{defect} — азимут от текущего положения пользователя к дефекту, θ — текущее направление движения.

Уведомление отправляется только если угол $\alpha \leq 45°$ (дефект находится в конусе обзора перед автомобилем).

Фильтрация и отправка уведомлений:

Поиск дефектов выполняется с использованием сетчатого индекса — определяются ячейки, попадающие в радиус поиска, и собираются все дефекты из этих ячеек. Отфильтровываются дефекты, не соответствующие настройкам пользователя, а также дефекты, уведомление о которых уже было отправлено в течение последней минуты. Для каждого подходящего дефекта вычисляется примерное время до столкновения:

$$t = \frac{\frac{d}{1000}}{\frac{v}{3600}} \tag{4}$$

(при $v < 1$ км/ч время не вычисляется).

Уведомление формируется с учётом уровня опасности — для критических дефектов используется более интенсивная вибрация.

Диаграмма последовательности оповещения водителя представлена на рисунке 7.

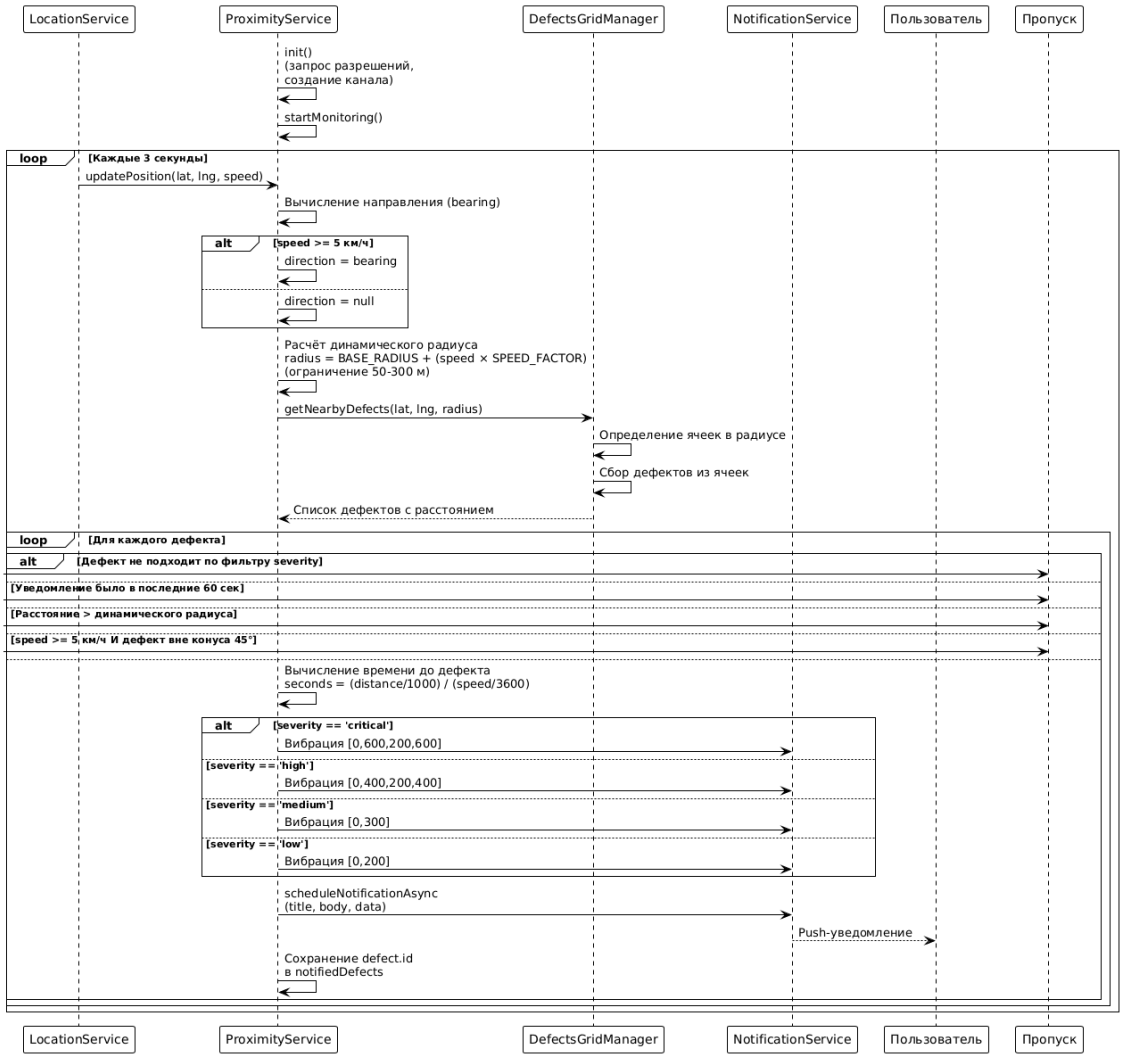

Рисунок 7 — Диаграмма последовательности оповещения водителя

2.3 Выводы

По результатам проектирования разработана архитектура системы информирования водителей о дефектах дорожного покрытия. Основные результаты проектирования:

1. Выбрана микросервисная архитектура, включающая сервис управления доступом, сервис управления дефектами, клиентское приложение, обратный прокси NGINX и подсистема мониторинга. Данный подход обеспечивает независимое масштабирование компонентов и устойчивость к отказам.
2. Для реализации серверной части выбран веб-фреймворк FastAPI, который по результатам сравнительного анализа демонстрирует наилучшие показатели производительности. Для клиентского приложения выбран React Native, обеспечивающий

кроссплатформенную разработку с единой кодовой базой для iOS и Android. В качестве обратного прокси и балансировщика нагрузки выбран NGINX.

3. Для хранения данных выбраны: PostgreSQL для хранения пользователей, ролей и разрешений; PostgreSQL с расширением PostGIS для хранения геопространственных данных о дефектах; Redis для хранения сессий и кэширования; MinIO для хранения фотографий дефектов и картографических тайлов.
4. Разработана концептуальная и детальная модели данных. Для сервиса управления доступом спроектированы таблицы users, roles, permissions, user_roles, role_permissions и login_history. Для сервиса управления дефектами спроектирована таблица road_defects с пространственными полями и GiST-индексами для оптимизации геозапросов.
5. Разработаны и описаны ключевые алгоритмы: загрузка и отображение дефектов на карте с использованием сеточного индекса и кэширования; оповещение водителя с динамическим радиусом и проверкой направления движения (конус 45°). Для остальных алгоритмов (аутентификация, обновление токенов, проверка прав, создание дефекта, геопоиск) приведены диаграммы последовательности в приложениях А и Б.

Все принятые проектные решения направлены на обеспечение производительности, масштабируемости и надёжности разрабатываемой системы. Спроектированная архитектура полностью соответствует требованиям, сформулированным в главе 1, и является основой для реализации системы в главе 3.

3 Глава 3. Реализация и тестирование

На основе спроектированной архитектуры и принятых проектных решений, описанных в главе 2, была осуществлена практическая реализация всех компонентов системы. В данной главе описываются особенности программирования, конфигурирования и интеграции разработанных модулей, а также приводятся результаты тестирования и оценка эффективности реализованной системы.

3.1 Реализация сервиса управления доступом

Сервис управления доступом реализован на языке Python с использованием веб-фреймворка FastAPI. Для хранения данных используется PostgreSQL, для управления сессиями — Redis. Аутентификация построена на JWT-токенах с алгоритмом RS256, пароли хэшируются с помощью bcrypt.

Структура проекта: проект организован по принципам чистой архитектуры (Clean Architecture) с разделением на слои: domain (бизнес-сущности и правила), application (use cases), infrastructure (реализация репозиториев и сервисов) и presentation (API handlers). Такой подход обеспечивает независимость бизнес-логики от внешних зависимостей и упрощает тестирование.

Особенности реализации:

1. Для генерации и верификации JWT-токенов использована библиотека authx, настроенная на алгоритм RS256. Приватный и публичный ключи передаются через переменные окружения в base64-кодировке.
2. Сессии пользователей хранятся в Redis с TTL, равным сроку действия refresh-токена (7 дней). Для каждого пользователя поддерживается список активных сессий.
3. Регистрация пользователя автоматически назначает ему роль «user». При успешной регистрации на указанный email отправляется письмо с ссылкой для верификации, содержащей JWT-токен.

4. Управление ролями и разрешениями реализовано через RBAC (Role-Based Access Control). Администратор может создавать роли, назначать им разрешения и назначать роли пользователям.

Скриншоты документации API сервиса управления доступом вынесены в приложение В.

3.2 Реализация сервиса управления дефектами

Сервис управления дефектами реализован на FastAPI с использованием PostgreSQL и расширения PostGIS для работы с геоданными. Для хранения фотографий используется объектное хранилище MinIO.

Импорт OSM-данных: дорожная сеть импортируется из OSM-файлов (.osm.pbf) с помощью утилиты osm2pgsql. Импорт создаёт таблицы planet_osm_line, planet_osm_point, planet_osm_polygon и другие, содержащие геометрию дорог и объектов инфраструктуры. Процесс импорта автоматизирован через CLI-команды.

Привязка дефектов к дорожной сети: при создании дефекта координаты пользователя привязываются к ближайшей дороге с помощью пространственного запроса к таблице planet_osm_line. Используются функции PostGIS: ST_DWithin для поиска в радиусе и ST_ClosestPoint для вычисления ближайшей точки на дороге.

Хранение и загрузка фотографий: фотографии загружаются в MinIO через S3-совместимый API. Для каждого дефекта создаётся отдельная директория, фотографии получают уникальные имена. URL загруженных фотографий сохраняются в поле photos таблицы road_defects.

Пространственные запросы: для поиска дефектов в радиусе используется функция ST_DWithin, для поиска в видимой области карты — ST_Intersects с прямоугольным полигоном, созданным через ST_MakeEnvelope. GiST-индексы на полях snapped_geometry и original_geometry обеспечивают высокую производительность запросов.

Скриншоты документации API сервиса управления дефектами вынесены в приложение В.

3.3 Реализация клиентского приложения

Клиентское приложение разработано на React Native, поддерживает платформы iOS и Android (а также Web через React Native Web). Для отображения карты использована библиотека MapLibre GL, работающая через WebView. Картографическая подложка представлена в формате PMTiles и загружается из MinIO.

Аутентификация и управление сессиями: токены доступа и обновления хранятся в защищённом хранилище (SecureStore на мобильных платформах, localStorage на Web). Реализован механизм автоматического обновления access-токена через интерцептор Axios при получении ошибки 401.

Отображение карты и дефектов: карта инициализируется с начальными координатами, определёнными по геолокации пользователя. Дефекты загружаются с сервера при перемещении карты с использованием сеточного индекса и кэширования. Для отображения дефектов используются два слоя: точечные маркеры (с цветом в зависимости от уровня опасности) и линии для протяжённых дефектов. Реализована кластеризация маркеров при малом масштабе.

Добавление дефекта: пользователь переводит карту в режим рисования, кликами отмечает точки дефекта. Для точечного дефекта достаточно одной точки, для линейного — двух и более. При добавлении каждой точки выполняется привязка к дорожной сети через эндпоинт `/v1/snap-point`. После завершения рисования открывается форма с выбором типа дефекта, уровня опасности, описанием и добавлением фотографий.

Оповещения о приближении: сервис proximityService периодически (каждые 3 секунды) проверяет дефекты в динамическом радиусе с учётом скорости и направления движения. При обнаружении дефекта в конусе 45° отправляется push-уведомление.

Скриншоты основных экранов мобильного приложения представлены на рисунках 8–10.

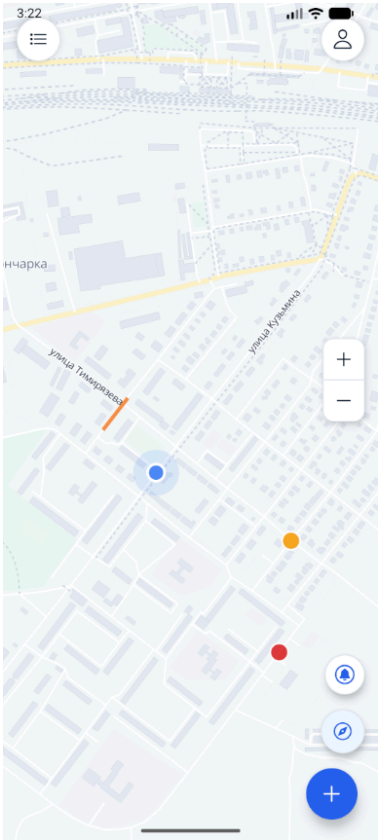

Рисунок 8 — Экран карты с дефектами

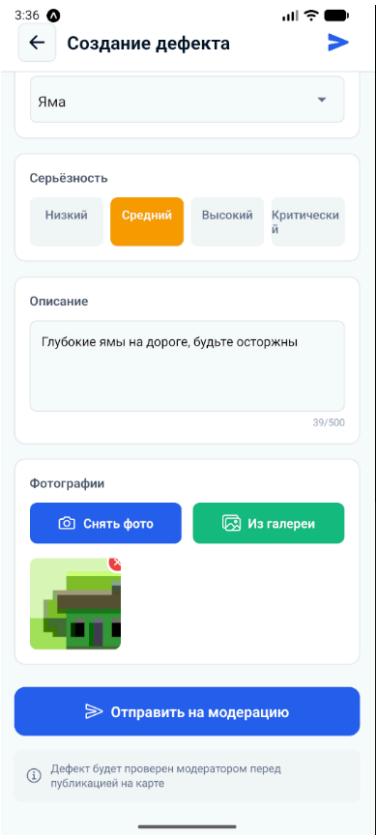

Рисунок 9 — Форма добавления дефекта

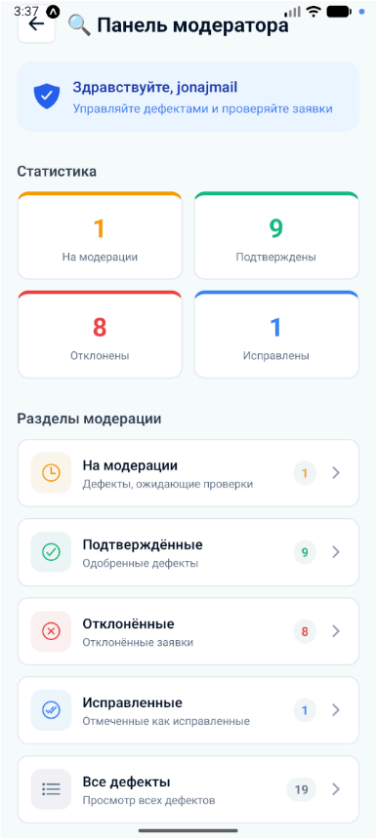

Рисунок 10 — Панель модератора

Остальные скриншоты интерфейса мобильного приложения (окно входа, регистрации, профиль, админ-панель, управление пользователями, ролями, разрешениями, списки дефектов, процесс добавления дефекта, оповещение) вынесены в приложение Г.

3.4 Тестирование системы

Тестирование разработанной системы включало модульное тестирование API, интеграционное тестирование пространственных запросов и функциональное тестирование клиентского приложения.

3.4.1 Тестирование API

Для тестирования API сервисов использовался фреймворк pytest. Были написаны тесты для проверки:

1. регистрации и аутентификации пользователей;
2. управления ролями и разрешениями;
3. создания, получения, обновления и удаления дефектов;

4. модерации дефектов;
5. пространственных запросов (поиск в радиусе и в области экрана).

Все тесты выполняются в изолированной тестовой базе данных, создаваемой заново перед каждым запуском. Результаты тестирования API представлены в таблице 8.

Таблица 8 — Результаты модульного тестирования API

Компонент	Количество тестов	Успешно
Сервис управления доступом	24	24
Сервис управления дефектами	18	18

3.4.2 Тестирование пространственных запросов

Для оценки производительности пространственных запросов было проведено нагрузочное тестирование с разным количеством дефектов в базе данных. Результаты представлены в таблице 9.

Таблица 9 — Результаты нагрузочного тестирования пространственных запросов

Количество дефектов	Поиск в радиусе, мс	Поиск в области, мс
1 000	23	18
10 000	45	32
50 000	89	67
100 000	156	112

3.4.3 Тестирование клиентского приложения

Функциональное тестирование клиентского приложения проводилось вручную на эмуляторах Android. Проверялись следующие сценарии:

1. вход и регистрация пользователя;
2. отображение карты и дефектов;
3. добавление нового дефекта (точечного и линейного);
4. просмотр и редактирование своих дефектов;
5. модерация дефектов (для модератора);
6. получение push-уведомлений при приближении к дефекту.

Все проверенные сценарии работают корректно. Выявленные в процессе тестирования ошибки были исправлены.

3.4.4 Результаты тестирования

По результатам тестирования можно сделать следующие выводы:

1. все модульные тесты API успешно пройдены;
2. пространственные запросы демонстрируют приемлемую производительность (до 156 мс при 100 000 дефектов);
3. клиентское приложение корректно отображает дефекты на карте и обрабатывает пользовательские сценарии;
4. система оповещений успешно обнаруживает дефекты в заданном радиусе и отправляет push-уведомления с учётом скорости и направления движения.

3.5 Выводы

В результате практической реализации были созданы и протестированы все компоненты системы:

1. Реализован сервис управления доступом на FastAPI с поддержкой JWT-аутентификации, управления сессиями через Redis и RBAC.
2. Реализован сервис управления дефектами на FastAPI с интеграцией PostGIS для пространственных запросов, привязкой дефектов к дорожной сети и хранением фотографий в MinIO.
3. Реализовано клиентское приложение на React Native с отображением карты (MapLibre GL + PMTiles), добавлением дефектов (рисование на карте) и оповещениями о приближении.
4. Проведено модульное и нагрузочное тестирование, подтвердившее работоспособность и производительность системы.

Таким образом, степень решения поставленных в главе 1 задач можно оценить как полную — все функциональные требования реализованы, нефункциональные требования (производительность, безопасность, масштабируемость) обеспечены.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была разработана информационная система информирования водителей о дефектах дорожного покрытия, основанная на краудсорсинговом принципе сбора данных от пользователей. Система обеспечивает оперативное выявление, модерацию, визуализацию дефектов и своевременное оповещение участников движения.

Достижение поставленной цели подтверждается решением всех сформулированных во введении задач.

По результатам аналитического обзора:

1. Определены основные типы дефектов дорожного покрытия, подлежащие фиксации в системе: трещины (продольные, поперечные, сетка, заделанные), выбоины, колейность, дефекты разметки, люки колодцев и заплатки.
2. Установлено, что дефекты дорожного покрытия оказывают существенное влияние на безопасность движения — по данным статистики, только в 2021 году из-за неудовлетворительного состояния дорог произошло почти 53 тысячи ДТП, в которых погибло 25 тысяч и ранено 68 тысяч человек.
3. Выявлены недостатки традиционных подходов к обнаружению дефектов (плановые осмотры, передвижные лаборатории, обращения граждан) и информированию водителей (дорожные знаки, навигационные приложения): неоперативность, неполнота данных, высокие затраты.
4. Проведён обзор существующих программных решений: Яндекс.Карты, 2ГИС, Waze, «Карта убитых дорог». Показано, что ни одно из них не сочетает детальную классификацию дефектов, оперативное оповещение и модерацию сообщений.
5. Выполнен анализ технологий и инструментов, применимых для разработки, включая архитектурные подходы, веб-фреймворки, СУБД, объектные хранилища, системы кэширования и мониторинга.
6. Сформулированы функциональные и нефункциональные требования к разрабатываемой системе, включая подсистемы

управления пользователями, управления дефектами, клиентского приложения и мониторинга.

По результатам проектирования:

1. Обоснован выбор микросервисной архитектуры, обеспечивающей независимое масштабирование компонентов и устойчивость к отказам.
2. Выбраны технологии реализации: FastAPI для серверной части, React Native для клиентского приложения, PostgreSQL с расширением PostGIS для геоданных, MinIO для хранения фотографий, Redis для кэширования и сессий, NGINX для маршрутизации и балансировки нагрузки.
3. Разработаны концептуальная и детальная модели данных, спроектированы ER-диаграммы для сервиса управления доступом и сервиса управления дефектами.
4. Разработаны и описаны ключевые алгоритмы: загрузка и отображение дефектов на карте с использованием сеточного индекса и кэширования; оповещение водителя с динамическим радиусом и проверкой направления движения (конус 45°).
5. Созданы диаграммы контекста, контейнеров в нотации C4 и диаграммы последовательности для всех ключевых сценариев использования.

По результатам реализации и тестирования:

1. Реализован сервис управления доступом на FastAPI с JWT-аутентификацией (RS256), управлением сессиями через Redis и RBAC. Проведено 24 модульных теста — все успешны.
2. Реализован сервис управления дефектами на FastAPI с интеграцией PostGIS для пространственных запросов, привязкой дефектов к дорожной сети (snap to road) и хранением фотографий в MinIO. Проведено 18 модульных тестов — все успешны.
3. Реализовано клиентское приложение на React Native с отображением карты (MapLibre GL + PMTiles), добавлением дефектов (точки и линии), кэшированием дефектов по сетке и оповещениями о приближении.

4. Проведено нагрузочное тестирование пространственных запросов: при 100 000 дефектов поиск в радиусе выполняется за 156 мс, поиск в области экрана — за 112 мс, что подтверждает эффективность GiST-индексов.

Научная новизна работы заключается в систематизации подходов к построению краудсорсинговых систем мониторинга дорожного покрытия, определении требований к их архитектуре и функциональности, а также в выявлении особенностей обработки геопространственных данных в контексте задач отображения на карте и информирования участников дорожного движения.

Практическая значимость работы состоит в разработке и реализации серверной части (сервисы управления доступом и управления дефектами) и клиентского мобильного приложения. Разработанная система может быть использована для оперативного информирования водителей о дефектах дорог, а также внедрена в качестве платформы для взаимодействия граждан с дорожными службами и органами контроля.

Направления дальнейшего развития системы:

1. внедрение автоматического обнаружения дефектов с использованием компьютерного зрения (анализ изображений с видеорегистраторов и дронов);
2. интеграция с государственными информационными системами для автоматической передачи подтверждённых дефектов в дорожные службы;
3. расширение системы аналитики — построение тепловых карт дефектов, прогнозирование износа дорожного покрытия;
4. внедрение системы лояльности и геймификации для стимулирования активности пользователей.

Таким образом, цель работы достигнута в полном объёме. Разработанная система готова к практическому использованию и может быть масштабирована для работы в любом регионе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ Р 50597-2017. Дороги автомобильные и улицы. Требования к эксплуатационному состоянию, допустимому по условиям обеспечения безопасности дорожного движения. Методы контроля. М., 2017. С. 28.
2. Сапрыкин С., Пак В. Влияние состояния дорожного полотна и дорожных условий на безопасность дорожного движения // Вестник магистратуры. 2022. № 3–1(126). Сс. 6–8.
3. Ричардсон К. Микросервисы. Паттерны разработки и рефакторинга. СПб.: Питер, 2019.
4. Ньюмен С. Создание микросервисов. СПб.: Питер, 2020.
5. Салтанов Д., Арсентьева Н. Сравнительный анализ архитектурных подходов к разработке REST API для высоконагруженных систем // Форум молодых учёных. 2025. № 6(106). Сс. 329–338.
6. Саенко А., Габов В., Журавлев А. Сравнительный анализ систем управления базами данных при реализации распределенного хранилища геоданных // Инженерный вестник Дона. 2025. № 11.
7. Amazon Web Services. Amazon S3 Documentation [Электронный ресурс]. 2025. URL: https://docs.aws.amazon.com/s3.
8. MinIO. MinIO Documentation [Электронный ресурс]. 2025. URL: https://min.io/docs.
9. Ceph Foundation. Ceph Documentation [Электронный ресурс]. 2025. URL: https://docs.ceph.com/.
10. Левин А., Семичев С., Казакова И. Обзор рынка in-memory систем управления базами данных // Вестник Пензенского государственного университета. 2024. № 4. Сс. 87–89.
11. Prometheus. Prometheus Documentation [Электронный ресурс]. 2025. URL: https://prometheus.io/docs.
12. Grafana Labs. Grafana Documentation [Электронный ресурс]. 2025. URL: https://grafana.com/docs.

ПРИЛОЖЕНИЕ А

Алгоритмы сервиса управления доступом

Диаграмма последовательности входа пользователя представлена на рисунке А.1.

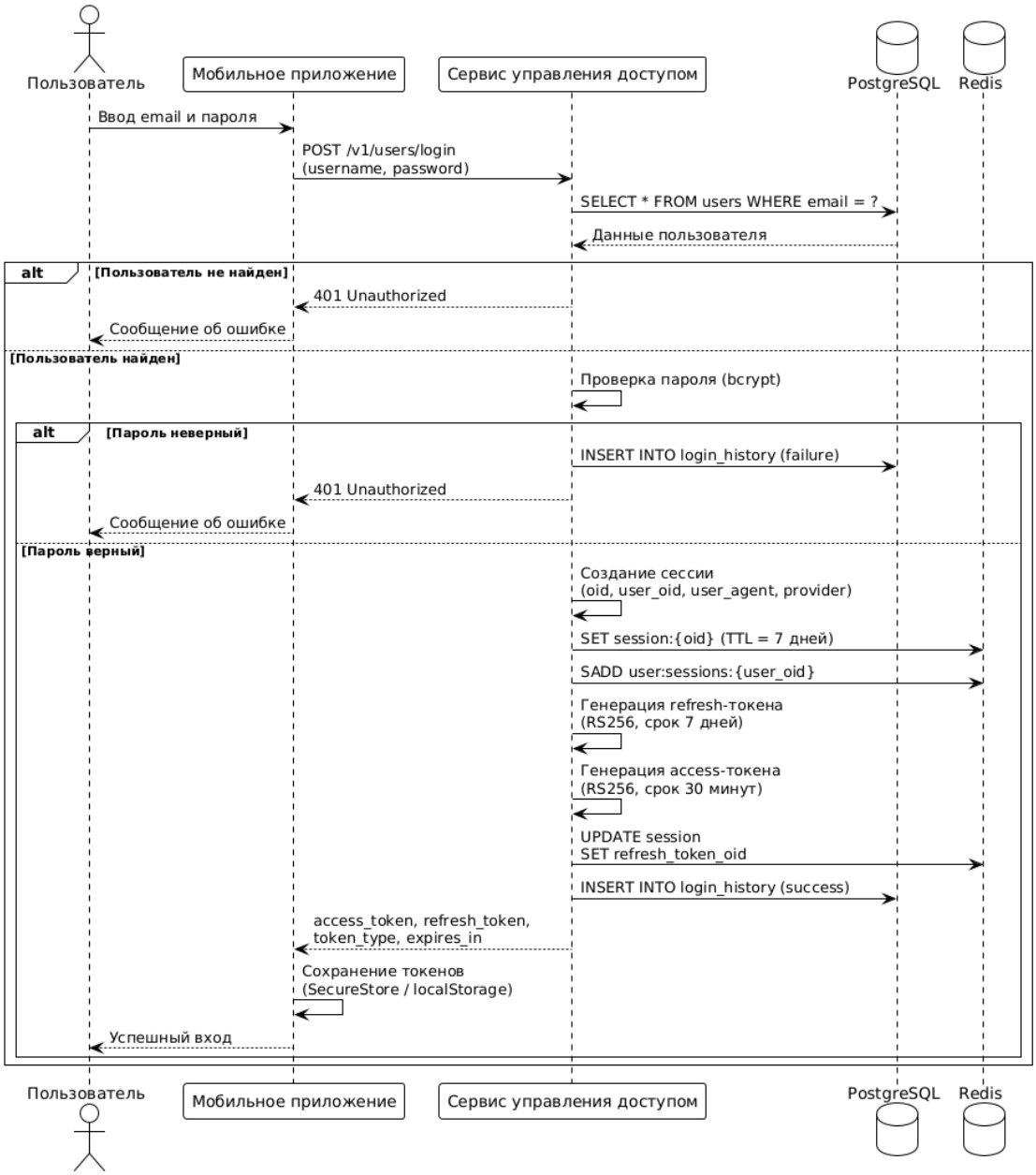

Рисунок А.1 — Диаграмма последовательности входа пользователя

Диаграмма последовательности обновления токенов представлена на рисунке А.2.

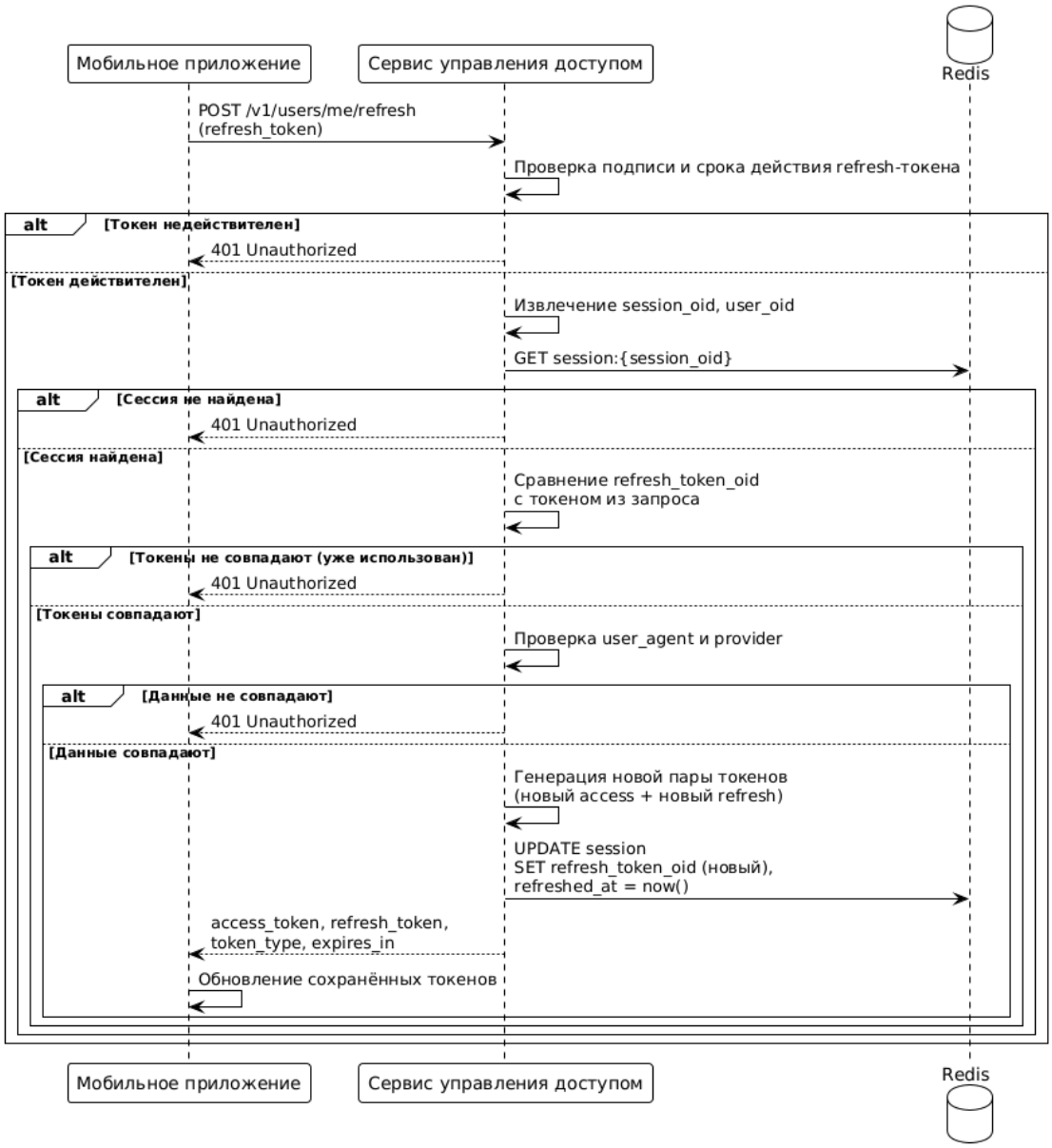

Рисунок А.2 — Диаграмма последовательности обновления токенов

Диаграмма последовательности проверки прав доступа представлена на рисунке А.3.

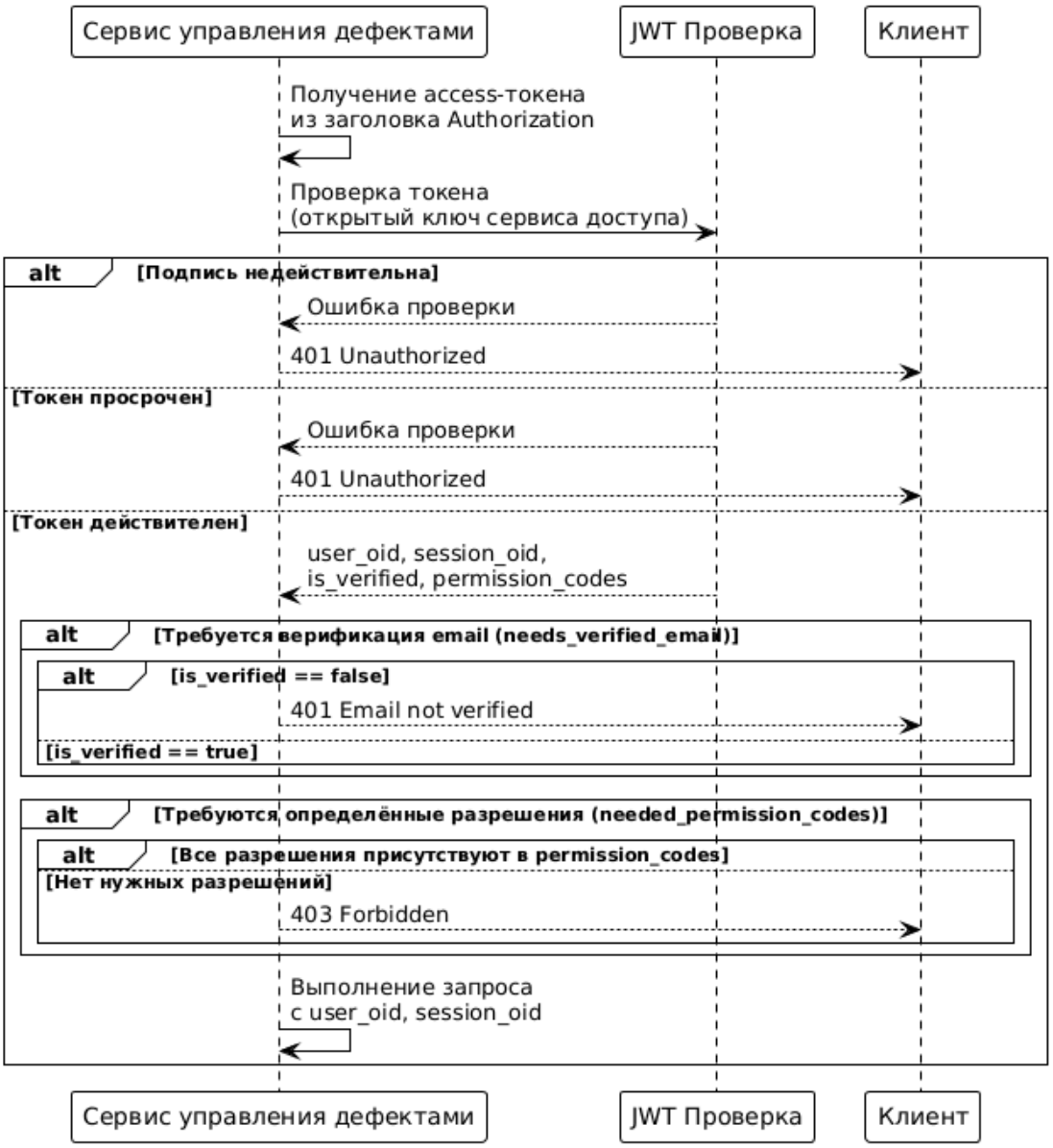

Рисунок А.3 — Диаграмма последовательности проверки прав доступа

ПРИЛОЖЕНИЕ Б

Алгоритмы сервиса управления дефектами

Диаграмма последовательности создания дефекта представлена на рисунке Б.1.

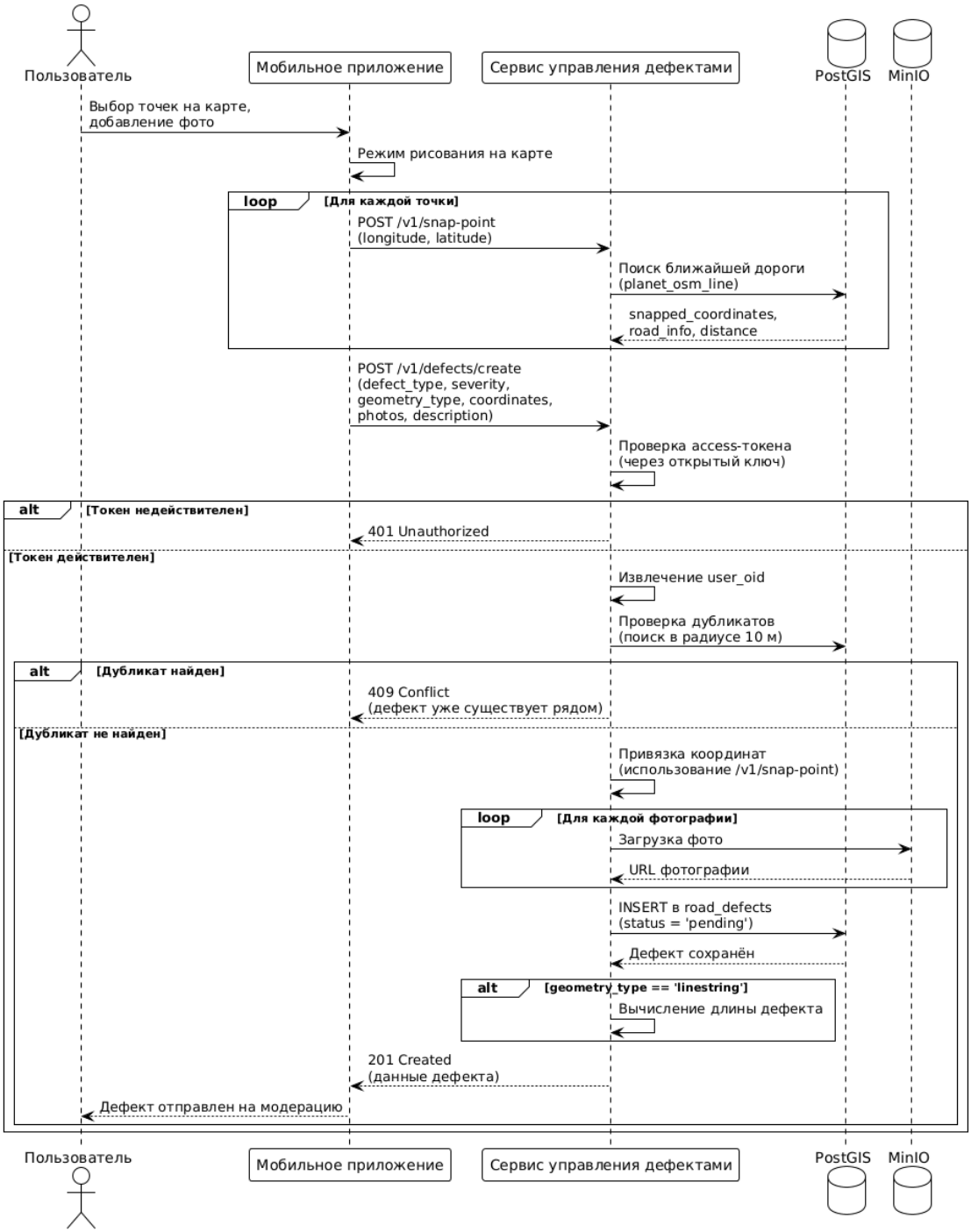

Рисунок Б.1 — Диаграмма последовательности создания дефекта

Диаграмма последовательности поиска дефектов рядом с местоположением представлена на рисунке Б.2.

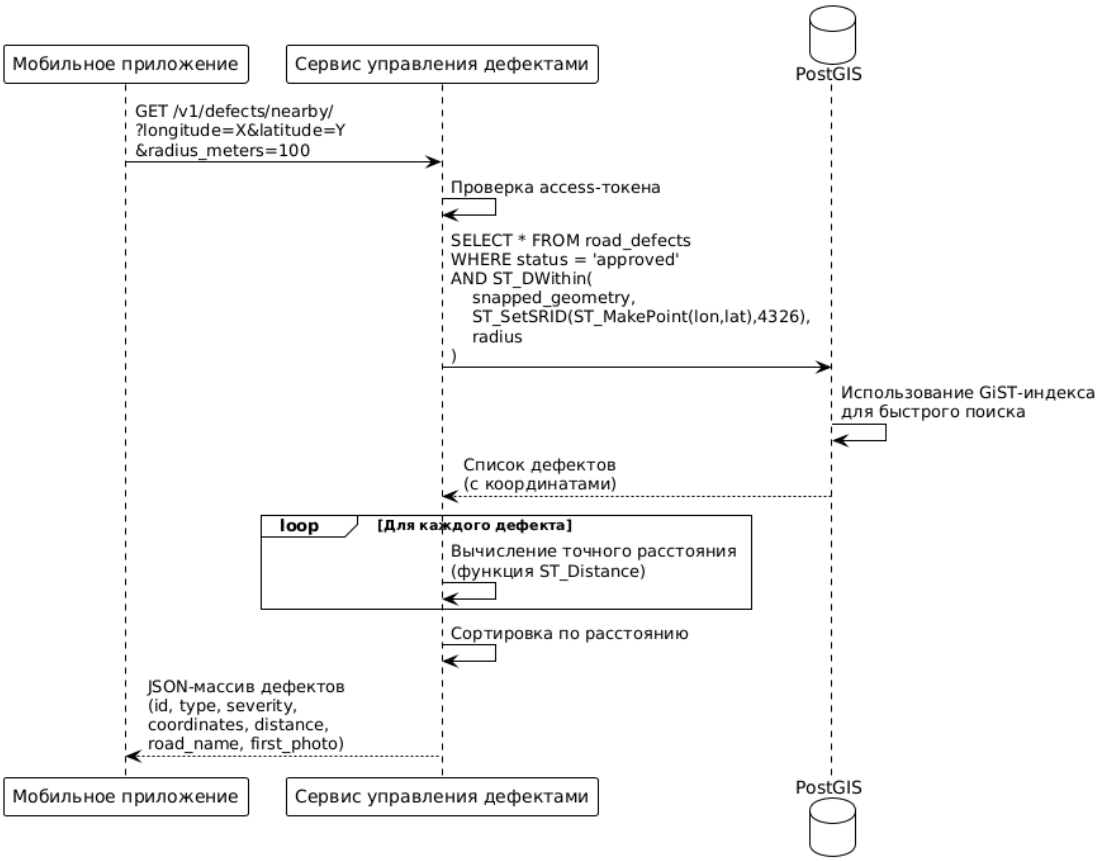

Рисунок Б.2 — Диаграмма последовательности поиска дефектов рядом с местоположением

Диаграмма последовательности поиска дефектов в видимой области карты представлена на рисунке Б.3.

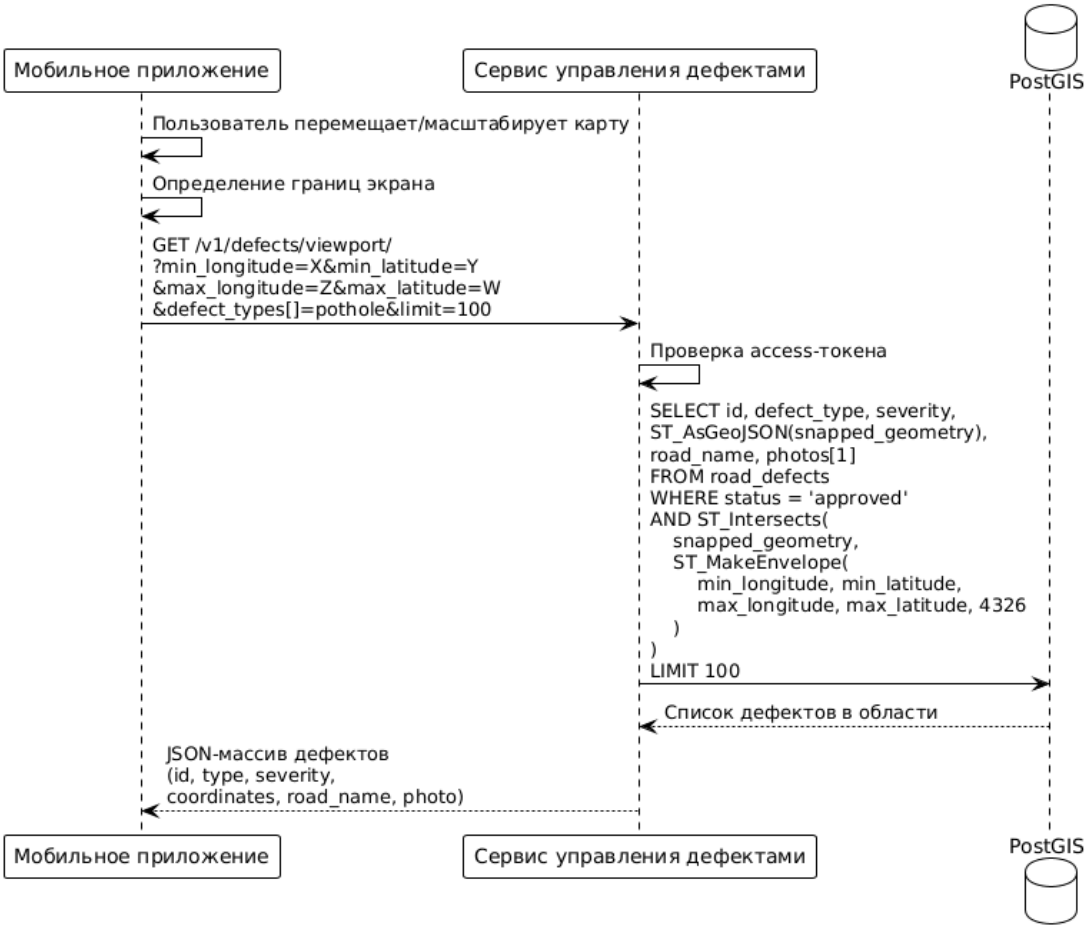

Рисунок Б.3 — Диаграмма последовательности поиска дефектов в видимой области карты

ПРИЛОЖЕНИЕ В

Swagger-документация API

Ниже представлены скриншоты интерактивной документации API (Swagger UI) разработанных микросервисов.

Сервис управления доступом: документация доступна по адресу `/api/docs` и содержит все эндпоинты для регистрации, аутентификации, управления сессиями, ролями и разрешениями.

Auth Service 0.1.0 OAS 3.1

/openapi.json

Auth Service

Authorize

user

POST /v1/users/register Register User

POST /v1/users/login Login User

POST /v1/users/me/refresh Refresh Token

DELETE /v1/users/me/logout Logout User

DELETE /v1/users/me/logout-all Logout All User

GET /v1/users Get Users

GET /v1/users/me Get Current User

GET /v1/users/me/session Get Current User Session Oids

GET /v1/users/{identifier} Get User By Identifier

POST /v1/users/{user_identifier}/roles/{role_identifier} Assign Role To User

DELETE /v1/users/{user_identifier}/roles/{role_identifier} Revoke Role From User

POST /v1/users/me/verify-by-email Email Verify By Email

GET /v1/users/me/verify Verify User

PATCH /v1/users/{identifier}/activate Activate User

PATCH /v1/users/{identifier}/deactivate Deactivate User

roles

POST /v1/roles/create Role Create

GET /v1/roles Roles Get List

GET /v1/roles/{identifier} Role Get By Identifier

PUT /v1/roles/{identifier} Role Update

DELETE /v1/roles/{identifier} Role Delete

POST /v1/roles/{role_identifier}/permissions/{permission_identifier} Role Assign Permission

DELETE /v1/roles/{role_identifier}/permissions/{permission_identifier} Role Assign Permission

permission

POST /v1/permission/create Role Create

GET /v1/permissions Permissions Get List

GET /v1/permissions/{identifier} Permission Get By Identifier

PUT /v1/permissions/{identifier} Permission Update

DELETE /v1/permissions/{identifier} Permission Delete

Рисунок B.1 — Swagger-документация сервиса управления доступом

Сервис управления дефектами: документация доступна по адресу `/api/docs` и содержит эндпоинты для создания, модерации, поиска дефектов, а также привязки координат к дорожной сети.

Defects Service 0.1.0 OAS 3.1

/openapi.json

Defects Service

Authorize

defects

POST /v1/defects/create Create Defect

GET /v1/defects/{defect_id} Get Defect

DELETE /v1/defects/{defect_id} Delete Defect

GET /v1/users/{user_id}/defects Get Defects By User Id

GET /v1/defects/nearby/ Get Defects Nearby

GET /v1/defects Get Defects

GET /v1/defects/viewport/ Get Defects In Viewport

GET /v1/defects/pending/ Get Pending Defects

PATCH /v1/defects/{defect_id}/moderate Moderate Defect

PATCH /v1/defects/{defect_id}/update Update Defect

roads

POST /v1/snap-point Snap Point

Рисунок В.2 — Swagger-документация сервиса управления дефектами

ПРИЛОЖЕНИЕ Г

Интерфейс мобильного приложения

Ниже представлены скриншоты основных экранов и диалоговых окон мобильного приложения.

Экран карты с дефектами: отображает карту с нанесёнными дефектами, маркеры имеют цвет в зависимости от уровня опасности. Реализована кластеризация маркеров на малых масштабах.

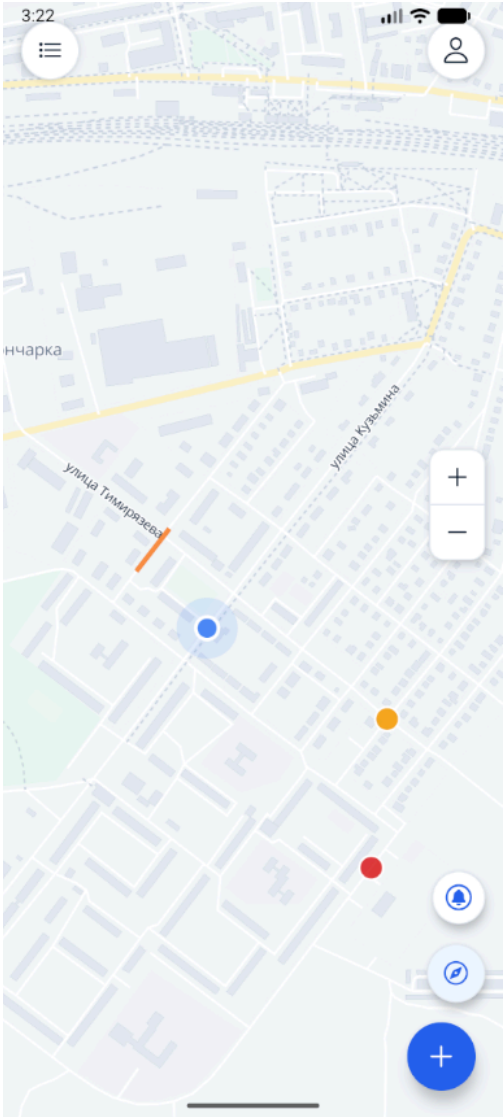

Рисунок Г.1 — Экран карты с дефектами

Попытка добавления дефекта без верификации: система запрашивает подтверждение email перед созданием дефектов.

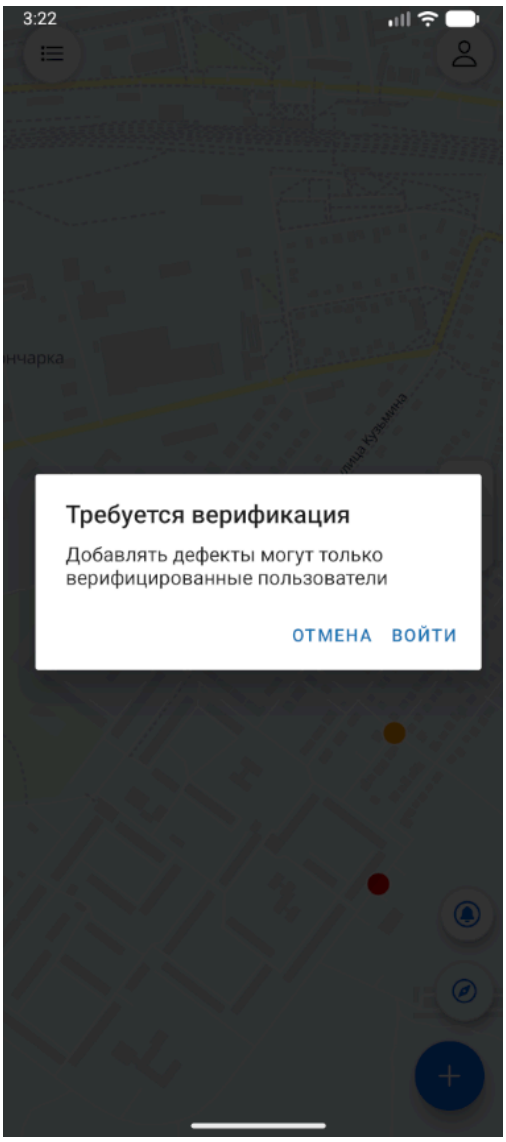

Рисунок Г.2 — Требование верификации email

Окно входа: форма для ввода email и пароля, кнопка входа и перехода к регистрации.

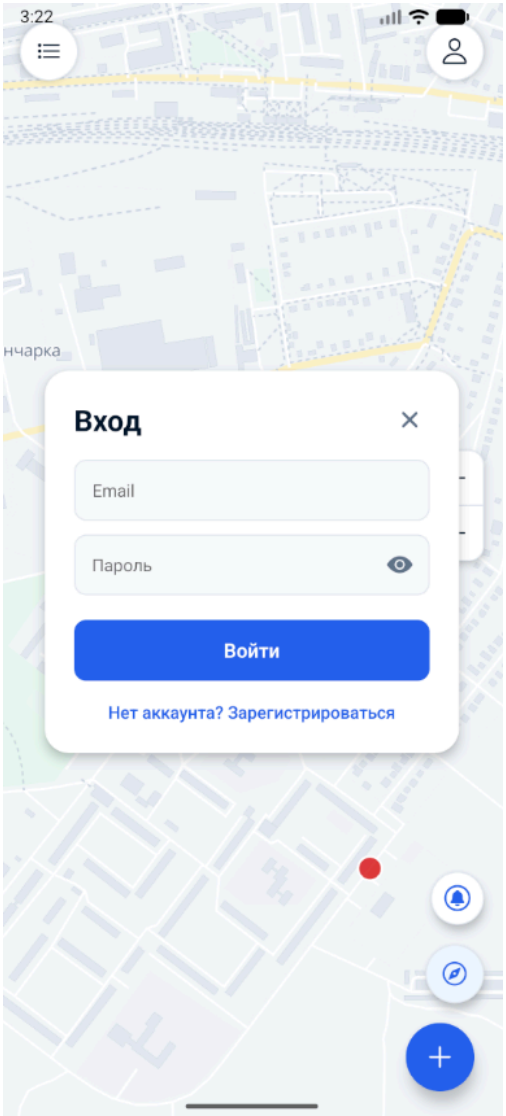

Рисунок Г.3 — Экран входа

Окно регистрации: форма для создания нового аккаунта.

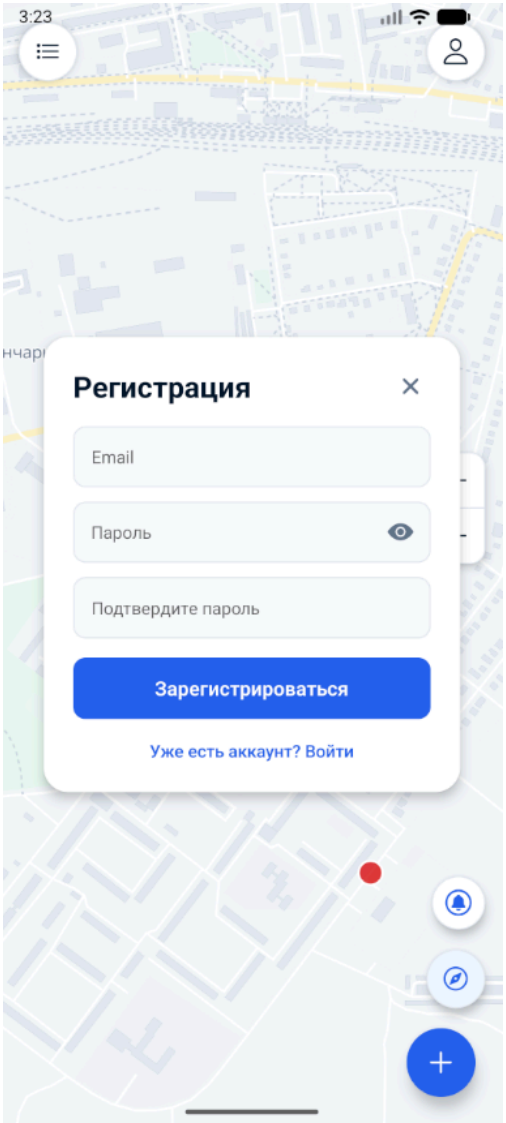

Рисунок Г.4 — Экран регистрации

Профиль пользователя: отображает информацию о пользователе, роли, сессии, кнопки выхода.

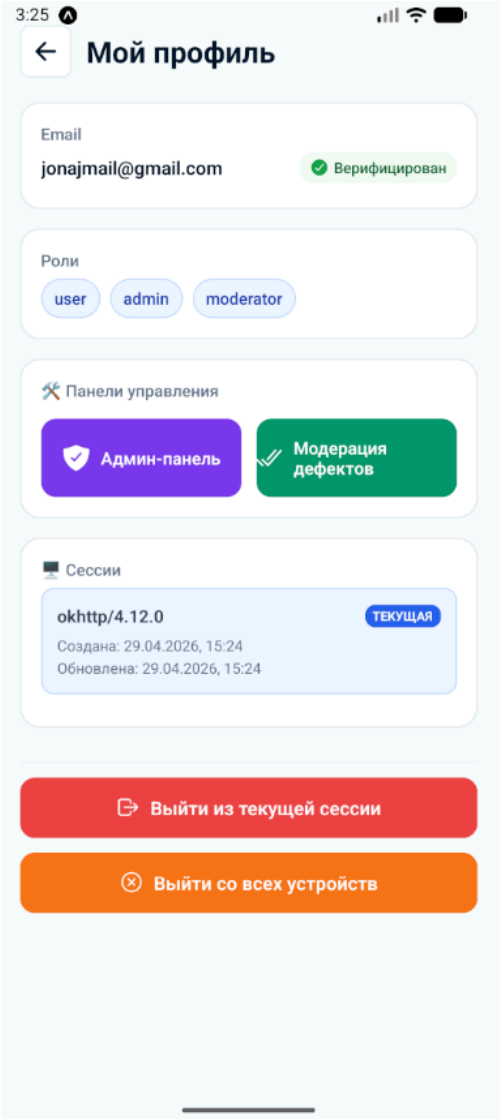

Рисунок Г.5 — Экран профиля

Admin-панель: панель управления для администратора (пользователи, роли, разрешения).

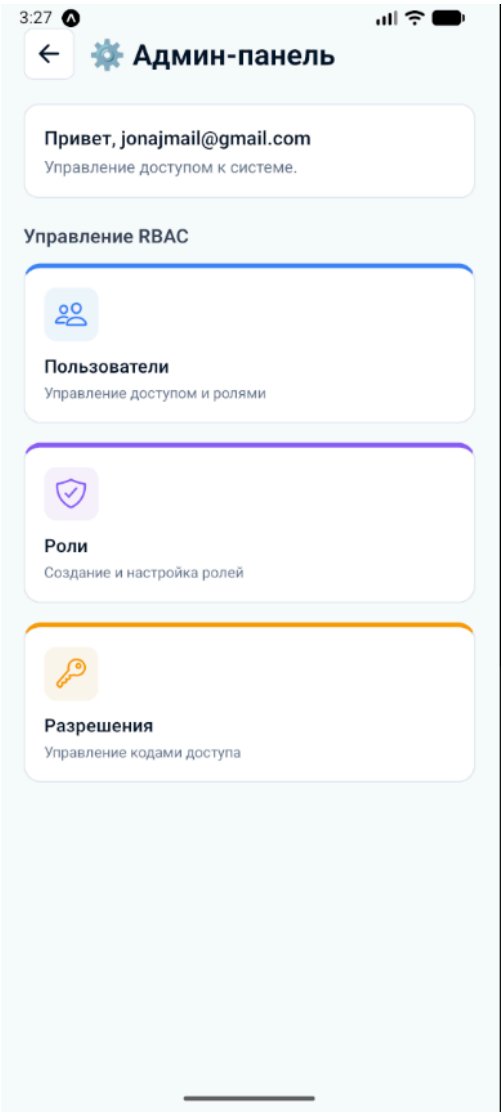

Рисунок Г.6 — Admin-panel

Список пользователей: таблица всех зарегистрированных пользователей с пагинацией.

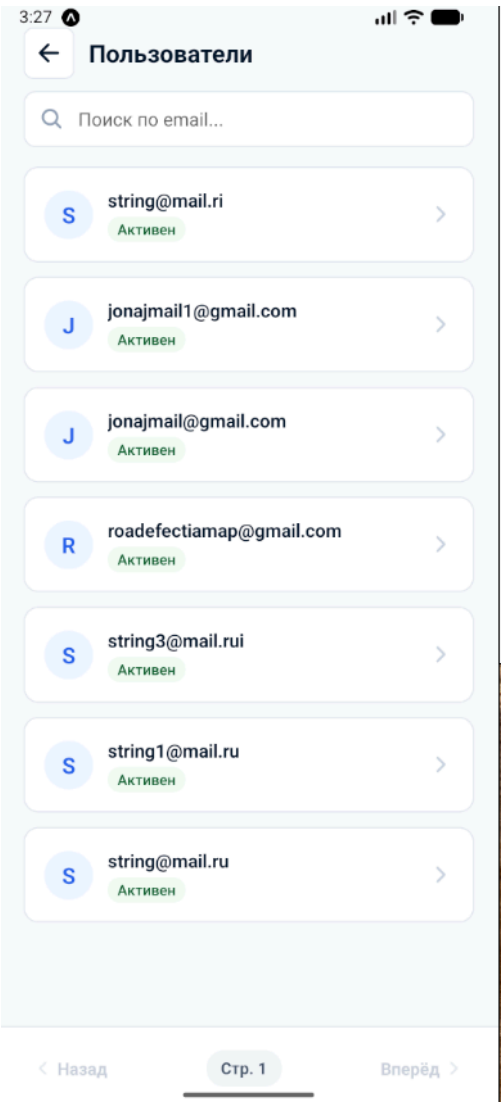

Рисунок Г.7 — Список пользователей

Управление пользователем: детальная информация о пользователе, возможность назначения ролей и активации/деактивации.

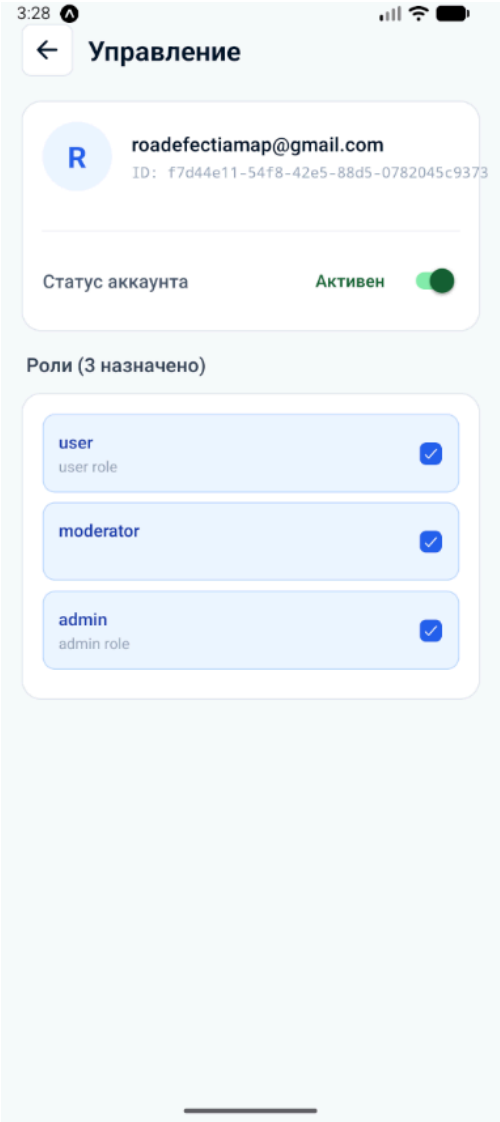

Рисунок Г.8 — Управление пользователем

Управление ролями: список ролей, возможность создания, редактирования и удаления.

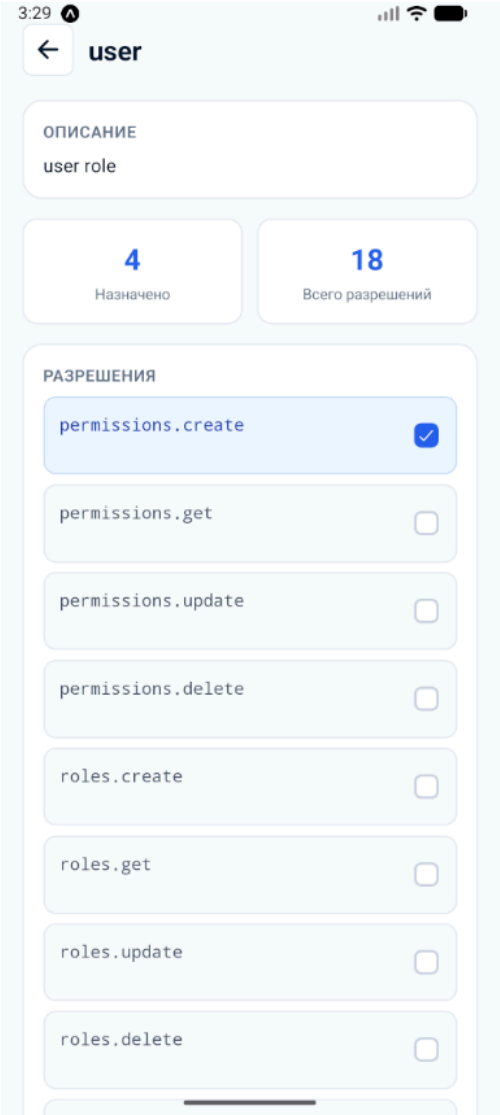

Рисунок Г.9 — Управление ролями

Управление разрешениями: список разрешений, возможность создания и удаления.

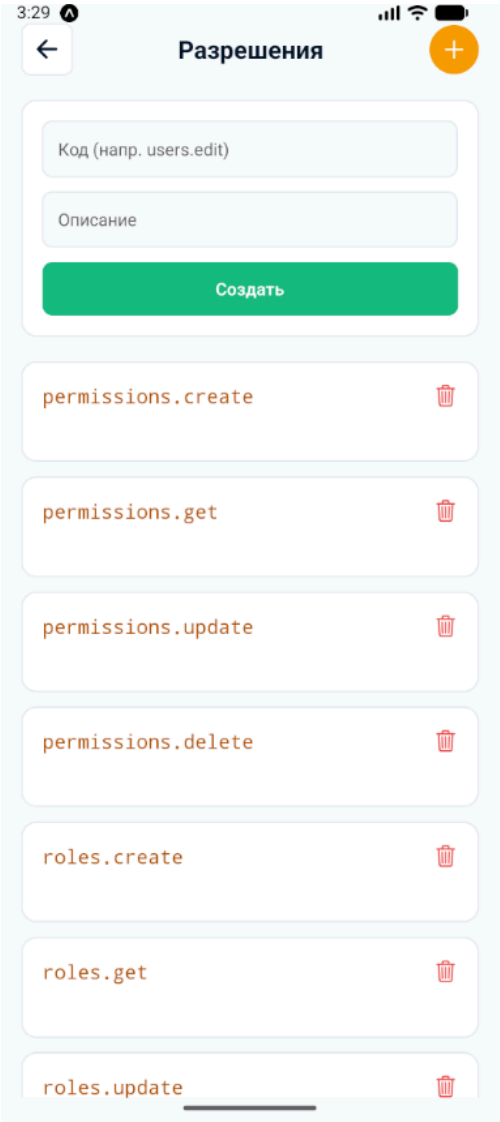

Рисунок Г.10 — Управление разрешениями

Список моих дефектов: отображает дефекты, созданные текущим пользователем, с фильтрацией по статусу.

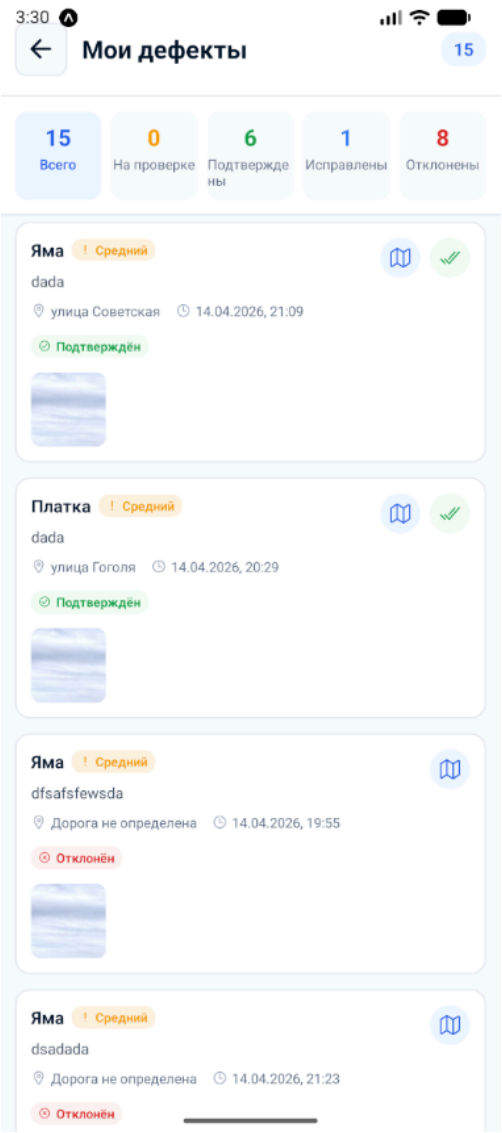

Рисунок Г.11 — Список моих дефектов

Режим рисования на карте: пользователь отмечает точки линейного дефекта на карте.

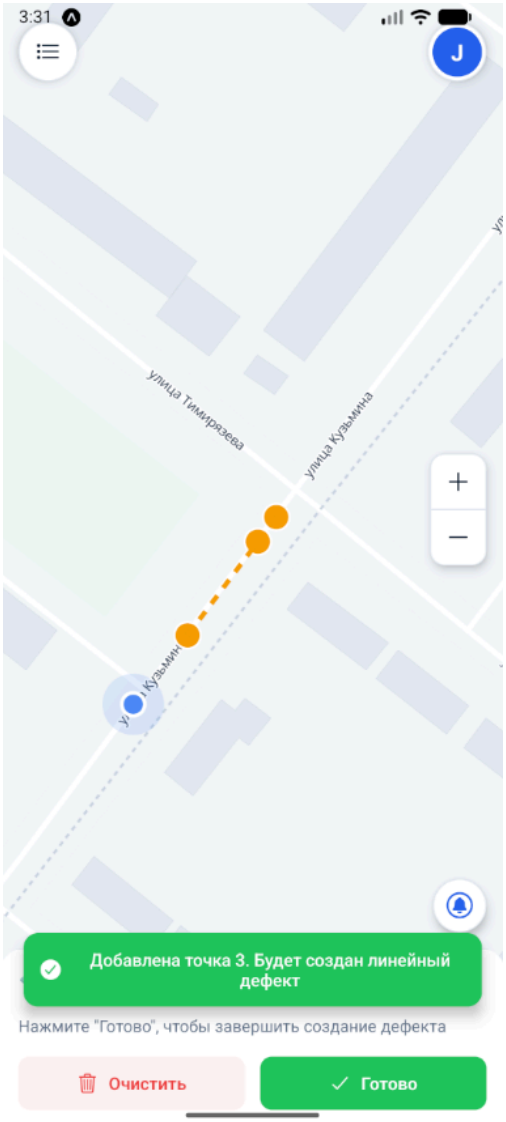

Рисунок Г.12 — Режим рисования дефекта на карте

Форма создания дефекта: выбор типа, уровня опасности, добавление описания и фотографий.

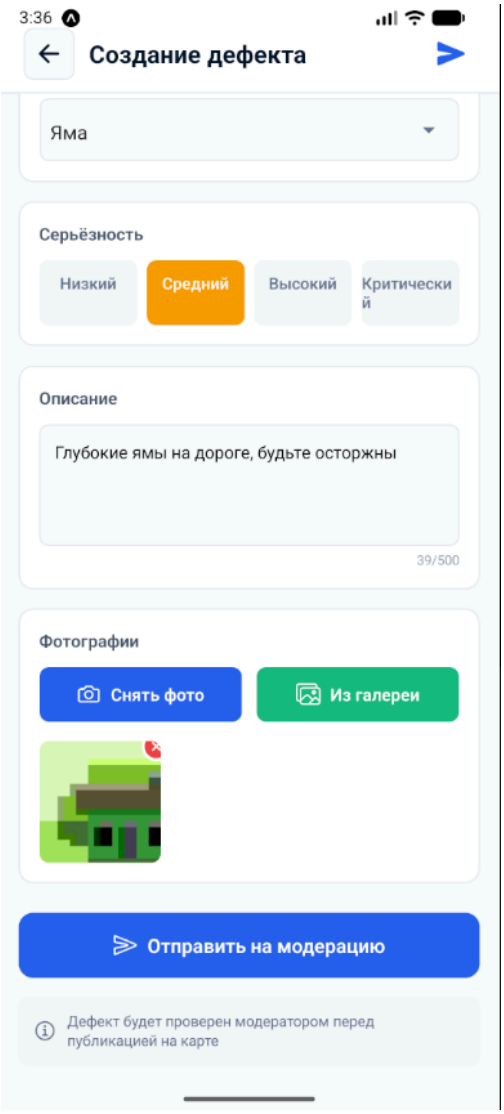

Рисунок Г.13 — Форма создания дефекта

Панель модератора: дашборд с количеством дефектов на модерации и разделами.

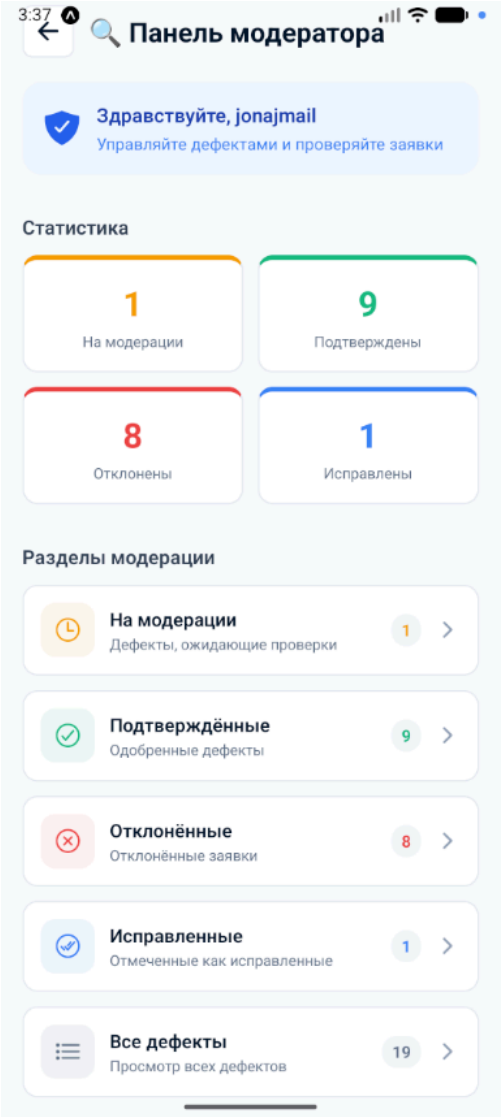

Рисунок Г.14 — Панель модератора

Модерация дефекта: просмотр дефекта с фотографиями, возможность подтвердить или отклонить с указанием причины.

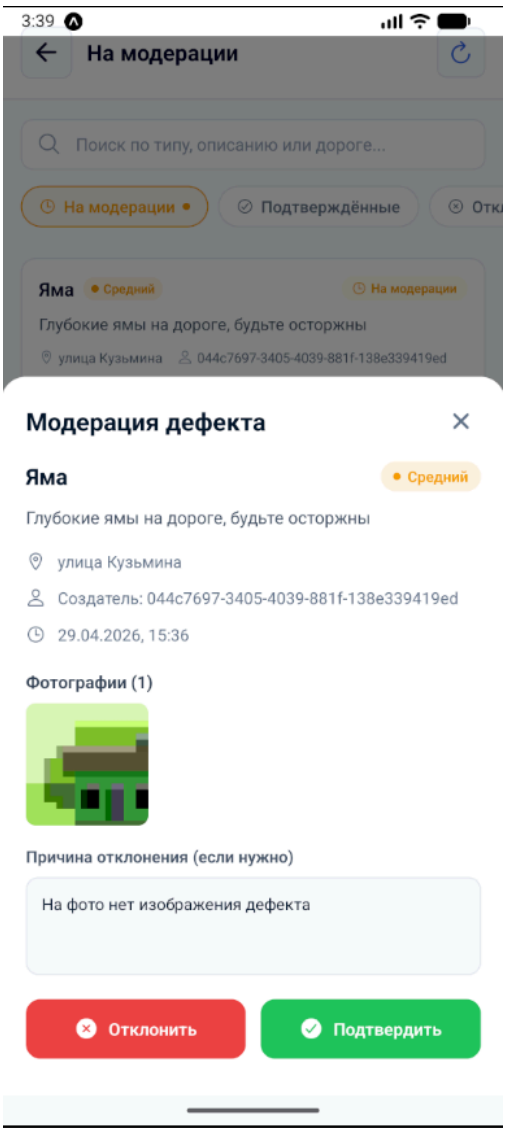

Рисунок Г.15 — Модерация дефекта

Оповещение о дефекте: push-уведомление при приближении к критическому или высокоопасному дефекту.

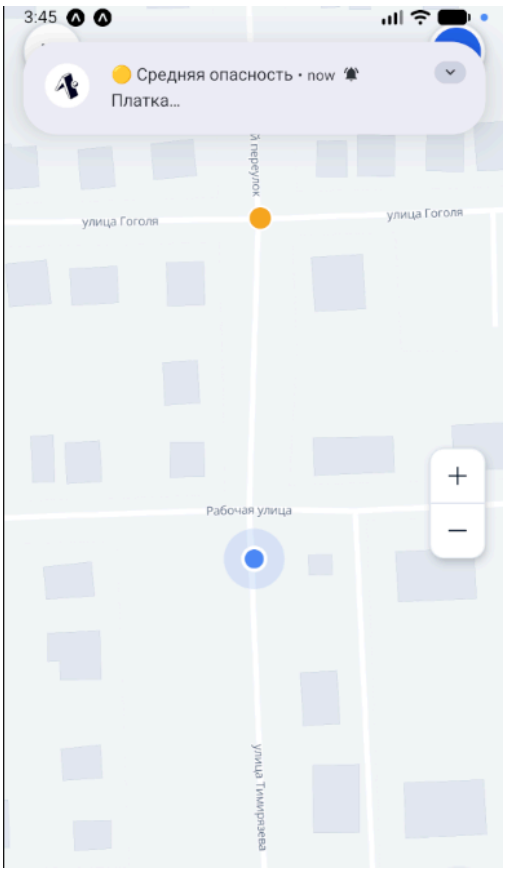

Рисунок Г.16 — Оповещение о приближении к дефекту